

Empirical Identification of a Variable-Structure Automaton under the P-Model

Engin Durmaz

Department of Computer Engineering
Faculty of Engineering, Marmara University
Istanbul, Turkey
engin.durmaz@live.com

Abstract

Learning-automata studies often require recovering the latent structure of a Variable-Structure Automaton (VSA)—policy, transitions, and penalty law—when only state–action–feedback interaction traces are observed. In a P-model environment the penalty probability is assumed to depend on the action but not on the source state; it remains unclear whether that assumption holds and whether estimates are stable under data loss. We address this setting with a fully empirical, count-based identification pipeline: validate the dataset, estimate normalized probabilities from sufficient statistics, identify an optimal action and ε -optimality gap, validate a forward simulator, stress-test estimates by random row removal, and audit per-state penalties $c_k(\phi_i)$ against the global P-model. The workflow is implemented as a reproducible end-to-end run on 3518 steps from 80 sequences. On the full dataset the lowest-penalty action is α_1 ($c_* \approx 0.20$), yet behavior is only weakly ε -optimal ($\varepsilon \approx 0.37$) and heterogeneous across states; a simulator matches empirical marginals within $L_1 \approx 0.016$. Under 20% subsampling (10 trials) the optimal action and penalty ranking are unchanged and ε moves little (0.371 ± 0.014), while per-state penalty spreads indicate moderate deviation from a strict action-only penalty law.

1 Introduction

Stochastic learning agents are frequently analyzed through Variable-Structure Automata (VSAs): finite-state controllers that select actions, update internal state, and receive binary environmental feedback. In the *P-model*, feedback $\beta \in \{0, 1\}$ indicates an unfavorable ($\beta = 1$) or favorable ($\beta = 0$) response, and a widely used identification target is the action-dependent penalty $c_k = P(\beta=1 \mid \alpha_k)$ together with the state–action policy $P(\alpha_k \mid \phi_i)$ and action-conditioned transitions $P(\phi_j \mid \phi_i, \alpha_k)$. A common working assumption is that c_k does not vary with the source state ϕ_i ; in practice, however, one must infer all of these quantities from observational interaction logs alone, assess whether behavior is nearly optimal with respect to the inferred penalties, and judge whether a generative simulator and a global penalty law are supported by the data.

This paper considers the inverse problem on a fixed interaction dataset with five states, three actions, 80 episodes, and 3518 recorded steps. Each observation records the sequence identifier, step index, source and destination states ($\phi_{\text{src}}, \phi_{\text{dst}}$), action α , and feedback β . Rather than fitting a parametric learner, we adopt a *nonparametric, count-based* scheme: empirical frequencies are accumulated, normalized where appropriate, and checked for internal consistency. The goals are to (i) validate the dataset and report sufficient statistics, (ii) estimate the VSA parameters and derived optimality quantities, (iii) compare simulated and empirical marginals, (iv) quantify sensitivity to random row removal, and (v) test the P-model via state-specific penalties $c_k(\phi_i)$.

The proposed workflow chains these stages in a single reproducible pipeline so that fingerprinting, estimation, ε -optimality analysis, simulation, perturbation, and model auditing share the same input and reporting conventions. Evaluation relies on explicit metrics— L_1 distances on marginals, the ε -optimality gap and spread of $P(\alpha^* \mid \phi_i)$, stability of α^* and ε under subsampling, and spread or drift of $c_k(\phi_i)$ relative to global c_k .

The main contributions are:

- An integrated empirical protocol that links validation, identification, generative checking, robustness testing, and critique of the P-model in one place.
- Explicit metrics at each stage so that results on the present dataset are comparable across runs and perturbation trials.
- A full empirical case study on the dataset above, including weak ε -optimality despite a clear low-penalty action, close agreement between simulated and empirical marginals, and stability of the optimal action under 20% row removal.
- Evidence that per-state penalty estimates exhibit non-trivial spread, so global c_k should be interpreted alongside $c_k(\phi_i)$ when assessing an action-only penalty law.

The remainder of this paper is organized as follows. Section 2 reviews the VSA/P-model setting and the count-based estimators. Section 3 describes the end-to-end identification pipeline and its implementation. Section 4 defines evaluation metrics, reports results, and discusses their interpretation. Section 5 summarizes the study. Appendix A provides supplementary source listings; Appendix B gives the complete experimental log.

2 Background

2.1 Variable-structure automata and the P-model

A **Variable-Structure Automaton** (VSA) is a stochastic decision model in which an agent visits discrete states ϕ_i , chooses actions α_k , and receives environment feedback. In this project the state set is $\{\phi_1, \dots, \phi_5\}$, the action set is $\{\alpha_1, \alpha_2, \alpha_3\}$, and each interaction step is recorded in the dataset together with a sequence identifier and step index. The learning goal is to recover, from many such sequences only, four objects required by the assignment:

- a **state-action policy** $P(\alpha_k \mid \phi_i)$,
- **transition dynamics** $P(\phi_j \mid \phi_i, \alpha_k)$,
- an **environmental penalty structure** (probabilities of unfavorable feedback), and
- a **norm of behavior** (e.g. how often the optimal action is taken in each state, via ε -optimality).

The environment is a **P-model**: feedback $\beta \in \{0, 1\}$ indicates a favorable (0) or unfavorable (1) response. The working assumption is that the *global* penalty $c_k = P(\beta=1 \mid \alpha_k)$ depends on the action but not explicitly on the source state—so a single global penalty vector c_k is often used for ranking actions and simulation. Estimating $c_k(\phi_i)$ separately in each state allows one to test whether penalties depend on ϕ_i beyond the action label.

2.2 Empirical estimators

All formulas below are evaluated on a (possibly subsampled) interaction table. Each CSV row is one step; we write $n \in R$ for row indices and S for the set of distinct `sequence_id` values. For a fixed sequence $s \in S$, J_s is how many rows belong to s (that sequence’s length in steps).

Dataset fingerprint. Before modeling, the raw log is summarized by five quantities:

- **Total number of rows** $|R|$ (one per step in the file).

- **Number of sequences** $|S|$ (distinct `sequence_id`).
- **Mean sequence length** $\bar{L} = \frac{1}{|S|} \sum_{s \in S} J_s$ (average of the per-sequence lengths J_s).
- **Empirical penalty rate per action** α_k : fraction of rows with $\alpha(n) = \alpha_k$ that have $\beta(n) = 1$,

$$\frac{1}{\#\{n \in R : \alpha(n) = \alpha_k\}} \sum_{\substack{n \in R \\ \alpha(n) = \alpha_k}} \beta(n).$$

- **Checksum** $C = \sum_{n \in R} \text{step}(n)$.

On the full dataset, each per-action rate above equals the global penalty c_k used in estimation (both are the sample mean of β given α_k).

Raw counts. Probabilities are derived from three count tables: the empirical basis for the policy $P(\alpha_k \mid \phi_i)$, the transitions $P(\phi_j \mid \phi_i, \alpha_k)$, and the penalties c_k . Each entry counts how many rows $n \in R$ match a pattern on `phi_src`, `phi_dst`, `alpha`, and `beta` (written $\Phi_{\text{src}}(n)$, $\Phi_{\text{dst}}(n)$, $\alpha(n)$, $\beta(n)$ below).

$N(\phi_i, \alpha_k)$ — **actions taken in a state.**

$$N(\phi_i, \alpha_k) = \#\{n \in R : \Phi_{\text{src}}(n) = \phi_i, \alpha(n) = \alpha_k\}. \quad (1)$$

$N(\phi_i, \alpha_k, \phi_j)$ — **transitions under an action.**

$$N(\phi_i, \alpha_k, \phi_j) = \#\{n \in R : \Phi_{\text{src}}(n) = \phi_i, \alpha(n) = \alpha_k, \Phi_{\text{dst}}(n) = \phi_j\}. \quad (2)$$

$N(\alpha_k, \beta)$ — **environment feedback per action.**

$$N(\alpha_k, \beta) = \#\{n \in R : \alpha(n) = \alpha_k, \beta(n) = \beta\}. \quad (3)$$

Write $r_i = \sum_k N(\phi_i, \alpha_k)$ for the row total over actions in state ϕ_i .

Conditional estimates. For visited states ($r_i > 0$),

$$P(\alpha_k \mid \phi_i) = \frac{N(\phi_i, \alpha_k)}{r_i}. \quad (4)$$

For each action α_k , transition probabilities normalize each row of $N(\phi_i, \alpha_k, \phi_j)$ over ϕ_j (empty rows yield zeros). The **global** P-model penalty is

$$c_k = P(\beta=1 \mid \alpha_k) = \frac{N(\alpha_k, 1)}{N(\alpha_k, 0) + N(\alpha_k, 1)}. \quad (5)$$

The action with lowest global penalty is $\alpha^* \in \arg \min_k c_k$. On visited states $\mathcal{V} = \{\phi_i : r_i > 0\}$, the ε -optimality gap is $\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \mid \phi_i)$. To simulate trajectories, sample $\alpha \sim P(\cdot \mid \phi)$, $\phi' \sim P(\cdot \mid \phi, \alpha)$, and $\beta \sim \text{Bernoulli}(c_\alpha)$ along each real sequence length.

Per-state penalties. To relax the action-only penalty assumption, for each pair (ϕ_i, α_k) with $N(\phi_i, \alpha_k) > 0$,

$$c_k(\phi_i) = P(\beta=1 \mid \alpha_k, \phi_i) = \frac{N(\phi_i, \alpha_k, \beta=1)}{N(\phi_i, \alpha_k)}, \quad (6)$$

where $N(\phi_i, \alpha_k, \beta=1)$ counts penalized steps with that state and action. Under a strict P-model, $c_k(\phi_i)$ should vary little in ϕ_i for fixed k ; a diagnostic is $\max_i c_k(\phi_i) - \min_i c_k(\phi_i)$ per action.

3 Methodology

The implementation comprises one module per task and is run end-to-end via `python main.py` $\rightarrow$ `load_all` $\rightarrow$ `run_all`. Formulas are in Section 2.2; Appendix A lists the matching source.

3.1 General flow

The end-to-end pipeline is implemented in `main.run_all` (Appendix A.6). It always validates the raw CSV, fits the VSA quantities on the full dataset, and optionally repeats the fit on random row subsamples to test stability. Per-state penalty tables are printed once at the end so the console log stays readable. Figure 1 summarizes `run_all` with decisions and loops; Algorithm 1 states the same logic in pseudocode.

Algorithm 1 General flow (`main.run_all`)

Require: CSV path, `perturbation_rate`, `experiment_count`, `random_seed`

```

1:  $df_{full} \leftarrow \text{load\_vsa\_dataset}(\text{path})$   $\triangleright$  util.py
2: run_fingerprint(path)  $\triangleright |R|, |S|, \bar{L}$ , penalty rates,  $C$ 
3:  $baseline \leftarrow \text{RUNTASKS1TO4}(df_{full}, baseline)$ 
4:  $results \leftarrow [baseline]$ 
5: if perturbation_rate  $> 0$  then
6:   for  $t = 1$  to experiment_count do
7:      $df_p \leftarrow \text{\_drop\_rows\_randomly}(df_{full}, \text{rate}, rng)$ 
8:     append RUNTASKS1TO4( $df_p, t$ ) to  $results$ 
9:   end for
10:  \_print\_experiments\_summary( $baseline$ , perturbed runs)
11: end if
12: \_print\_c\_k\_per\_state\_runs( $results$ )

13: function RUNTASKS1TO4( $df, label$ )
14:   $est \leftarrow \text{run\_estimation}(df, \text{print\_penalty\_per\_state}=\text{False})$ 
15:   $opt \leftarrow \text{run\_optimal\_action}(est)$ 
16:   $eps \leftarrow \text{run\_epsilon\_optimality}(est, opt)$ 
17:   $sim \leftarrow \text{run\_simulation}(est, path)$ 
18:  return PipelineRunResult( $label, est, opt, eps, sim$ )
19: end function

```

The implementation is **purely data-driven**: every quantity is an empirical frequency or a ratio of counts from the provided CSV, followed by row-wise normalization where conditional distributions are required. We do not fit parametric models, run gradient-based learning, or impose structure beyond what the counts support. Consistency checks (policy rows and transition rows sum to one; global c_k matches the sample mean of β given α) validate the arithmetic.

The six assignment tasks form the pipeline above: a **dataset fingerprint** (Section 2.2) validates scale and raw penalty rates; **estimation** builds counts and conditional probabilities; **optimal-action** and ε -**optimality** analyses use the policy and global penalties; **simulation** draws synthetic trajectories from the learned model and compares marginals to the real data; **perturbation** randomly deletes rows and refits to test robustness; **model critique** compares per-state penalties $c_k(\phi_i)$ across ϕ_i to assess the P-model. Subsection 3.2 maps each step to code; Appendix A lists the corresponding excerpts.

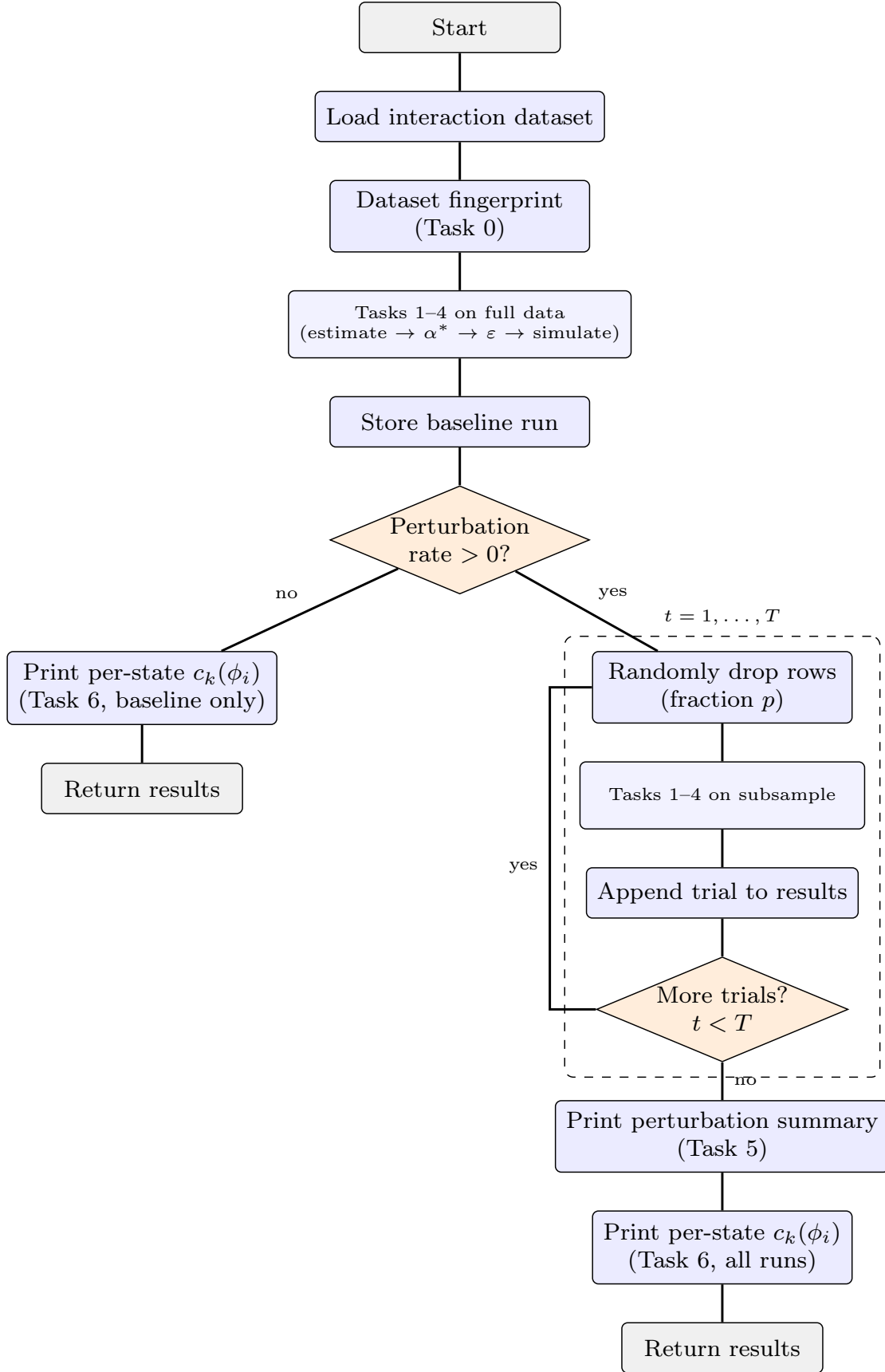


Figure 1: Flowchart for `run_all`: baseline Tasks 1–4 on the full dataset, optional perturbation loop over T trials (random row removal, refit, append), then Task 5 aggregate reporting when perturbation is enabled and Task 6 per-state penalty tables (baseline only if $p = 0$, else all stored runs).

3.2 Implementation details

The subsubsections below describe how each module realizes one part of the methodology. For each file we state the goal, point to the matching appendix listing, and list the main third-party libraries used (the standard library supplies `pathlib`, `dataclasses`, `argparse`, and `typing` where needed).

3.2.1 Software map

Table 1 lists each project module, its main entry point(s), return type, and the matching appendix listing (file by file, no task labels).

Table 1: Source modules in the Python implementation, as used by `main.run_all` via `python main.py`.

Module	Entry point(s)	Return type	Ref.
<code>fingerprint.py</code>	<code>run_fingerprint</code>	<code>Fingerprint-Result</code>	A.1
<code>estimation.py</code>	<code>run_estimation</code>	<code>Estimation-Result</code>	A.2
<code>optimal_action_identification.py</code>	<code>run_optimal_action</code>	<code>OptimalAction-Result</code>	A.3
<code>e_optimality_analysis.py</code>	<code>run_epsilon_optimality</code>	<code>Epsilon-Optimality-Result</code>	A.4
<code>simulation.py</code>	<code>run_simulation</code>	<code>Simulation-Result</code>	A.5
<code>main.py</code>	<code>run_all</code> , <code>load_all</code>	<code>PipelineRun-Result (list)</code>	A.6
<code>util.py</code>	<code>load_vsa_dataset</code>	<code>DataFrame</code>	A.7

3.2.2 Dataset fingerprint (`fingerprint.py`)

Goal. Before modeling, report $|R|$, $|S|$, $\bar{L}$, empirical penalty rate per action, and checksum C (Section 2.2) so the submission can be validated against the assignment fingerprint.

Code. Appendix A.1 (`run_fingerprint`, lines 86–99), invoked at the start of Algorithm 1.

Libraries. `pandas` (`groupby`, `mean`, `sum`).

3.2.3 Estimation (`estimation.py`)

Goal. From the interaction table, build raw count matrices and derive $P(\alpha_k \mid \phi_i)$, $P(\phi_j \mid \phi_i, \alpha_k)$, global c_k , and per-state $c_k(\phi_i)$; run consistency checks (Section 2.2).

Code. Appendix A.2: table formatters with `SUM/Cons.` columns (lines 65–128), count/probability estimation and checks (lines 206–276), and report printing (lines 296–315). From `main`, per-state penalties are computed but not printed (`print_penalty_per_state=False`). Optional CSV output is written under `outputs/task1_estimation/`.

Libraries. `pandas` (`groupby`, `unstack`, `normalization`); `NumPy` (`allclose` for checks).

3.2.4 Optimal action (`optimal_action_identification.py`)

Goal. Identify $\alpha^* \in \arg \min_k c_k$ from the penalty vector and record ties deterministically.

Code. Appendix A.3 (`run_optimal_action`, lines 109–128); uses `estimation.penalty` in memory or `penalty.csv` on disk.

Libraries. `pandas` (sorting, framing c_k); `NumPy` (`isclose` for tie detection).

3.2.5 ε -optimality (`e_optimality_analysis.py`)

Goal. Compute $\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \mid \phi_i)$ over visited states only.

Code. Appendix A.4 (`_compute_core` / `run_epsilon_optimality`, lines 110–130).

Libraries. `pandas` (policy and count matrices); `NumPy` (`min`, `isclose`).

3.2.6 Simulation (`simulation.py`)

Goal. Generate synthetic trajectories with the learned policy, transitions, and Bernoulli penalties; compare data vs. simulated marginals and penalty rates.

Code. Appendix A.5 (`_simulate_block`, `run_simulation`, lines 134–184).

Libraries. `pandas` (marginals, alignment); `NumPy` (`default_rng`, sampling, binomial draws).

3.2.7 Perturbation experiment (`main.py`)

Goal. Delete a random fraction of rows, refit the model on each subsample, and compare α^* , ε , and simulation L1 to the baseline (Algorithm 1, perturbation branch).

Code. Appendix A.6: `_drop_rows_randomly` (lines 36–51), `run_all` perturbation loop (lines 319–344), `_print_experiments_summary` (lines 162–255 in Appendix A.6).

Libraries. `NumPy` (`default_rng`, choice for row indices); `pandas` (`drop`, `reset_index`).

3.2.8 Model assumption check (`estimation.py`, `main.py`)

Goal. Form $c_k(\phi_i)$ for each state–action pair and compare across states (spread diagnostic); optionally across perturbation trials (Section 2.2).

Code. `penalty_per_state` in Appendix A.2; printing and spread helpers in Appendix A.6 (`_spread_across_states`, `_print_c_k_per_state_runs`, lines 70–78, 150–159).

Libraries. `pandas` (per-state table); `NumPy` (`max`/`min` spread, absolute differences). Also see `_run_tasks_1_to_4` and `run_all` in Appendix A.6 (lines 91–133, 258–348).

3.2.9 Shared data loading (`util.py`)

Goal. Load the headerless interaction CSV into a typed `pandas.DataFrame` and define shared result containers passed between modules.

Code. Appendix A.7: column constants, `EstimationResult`, `OptimalActionResult`, and `load_vsa_dataset`.

Libraries. `pandas` (`read_csv`, frame types).

4 Experiments

This section reports the empirical evaluation of the inferred VSA on the course dataset (assignment Tasks 1–6). A single run of `main.py` (`load_all`) fits the full log ($n = 3518$) and repeats Tasks 1–4 on ten subsamples with 20% random row removal (`random_seed=42`). Section 4.1 defines the metrics; Sections 4.2 and 4.3 present and interpret the outcomes; the complete console trace is in Appendix B.

4.1 Evaluation metrics and methods

This subsection records how each assignment task is evaluated before the numerical results in Section 4.2. Table 2 summarizes the evaluation metrics and criteria for assignment Tasks 1–6.

Table 2: Evaluation metrics and evaluation criteria (assignment Tasks 1–6).

Task	Metric	How evaluated
1	Raw counts	Report $N(\phi_i, \alpha_k)$, $N(\phi_i, \alpha_k, \phi_j)$, $N(\alpha_k, \beta)$ by counting matching rows (Section 2.2).
1	Policy	$P(\alpha_k \phi_i) = N(\phi_i, \alpha_k) / r_i$ for $r_i > 0$.
1	Transition model	$P(\phi_j \phi_i, \alpha_k)$: normalize each row of $N(\phi_i, \alpha_k, \phi_j)$ over ϕ_j .
1	Global penalty	$c_k = P(\beta=1 \alpha_k) = N(\alpha_k, 1) / (N(\alpha_k, 0) + N(\alpha_k, 1))$.
1	Consistency check (policy)	For each visited ϕ_i ($r_i > 0$): verify $\sum_k P(\alpha_k \phi_i) = 1$ (PASS/FAIL).
1	Consistency check (transition)	For each (ϕ_i, α_k) with transition mass: verify $\sum_j P(\phi_j \phi_i, \alpha_k) = 1$ (PASS/FAIL).
1	Consistency check (penalty)	Verify computed c_k equals the empirical fraction of $\beta=1$ among rows with action α_k (PASS/FAIL).
2	Optimal action	$\alpha^* \in \arg \min_k c_k$; if several actions tie at the minimum, report the first in sorted action order.
2	Minimal penalty	$c_* = \min_k c_k$.
3	ε -optimality gap	$\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \phi_i)$ with $\mathcal{V} = \{\phi_i : r_i > 0\}$ (never-visited states excluded).
3	Interpretation	Answer the assignment prompts (strong optimality, noise, cross-state consistency) from ε and the spread of $P(\alpha^* \phi_i)$ on $\mathcal{V}$.
4	Synthetic data	Generate sequences with the learned policy and transitions; sample $\beta \sim \text{Bernoulli}(c_\alpha)$; match real row count (fixed seed).
4	Action distribution	$ P_{\text{data}}(\alpha) - P_{\text{sim}}(\alpha) $ per action; report $L_1^{\text{act}} = \sum_\alpha \cdot $.
4	State visitation	$ P_{\text{data}}(\phi) - P_{\text{sim}}(\phi) $ for ϕ_{src} and ϕ_{dst} marginals; report L_1 sum over states for each.
4	Penalty rates	$ \beta_{\text{data}} - \beta_{\text{sim}} $ globally and per action (β = sample mean of β).
5	Perturbed refit	Remove $\approx 20\%$ of rows at random; recompute c_k , policy, and ε (Tasks 1–3) on each subsample.
5	Stability	Compare α^* , $\{c_k\}$, ε , and simulation L_1^{act} to baseline; summarize range, mean, and std across trials.
6	Per-state penalty	$c_k(\phi_i) = P(\beta=1 \alpha_k, \phi_i)$ from counts with $N(\phi_i, \alpha_k) > 0$ (Section 2.2).
6	Cross-state comparison	For each k , spread = $\max_{\phi_i} c_k(\phi_i) - \min_{\phi_i} c_k(\phi_i)$; inspect full $c_k(\phi_i)$ table to assess whether penalties depend on state.

Input: the full dataset ($|R| = 3518$, $|S| = 80$, $\bar{L} = 43.98$). One orchestrated run (`python main.py`) validates the fingerprint, fits Tasks 1–4 on the full table (baseline), then refits on ten subsamples with 704 rows ($\approx 20\%$) removed uniformly at random without replacement (`random_seed=42`). Count and probability formulas are in Section 2.2; code mapping is in Section 3.

4.2 Experimental results

Per-trial count matrices and full Task 1–4 printouts are omitted here (Appendix B).

4.2.1 Baseline (full data)

Fingerprint checksum $C = 83035$ matches the assignment reference. Global penalties are ordered $c_1 < c_2 < c_3$ with $c_1 = 0.200$, $c_2 = 0.382$, $c_3 = 0.518$, so $\alpha^* = \alpha_1$ and $c_* = 0.200$. The empirical policy is biased toward α_1 in every state ($P(\alpha_1 | \phi_i) \in [0.63, 0.83]$), yet $\varepsilon = 0.370$ because ϕ_5 still chooses α_1 only about 63% of the time—behavior is *weakly* optimal and heterogeneous across states (spread 0.201 in $P(\alpha^* | \phi_i)$). Simulation tracks the data well: $L_1 = 0.016$ on actions,

0.015 on ϕ_{src} , 0.016 on ϕ_{dst} , and $|P(\beta=1)_{\text{data}} - P(\beta=1)_{\text{sim}}| = 0.0077$ globally. Tables 3 and 4 summarize the baseline checks and main quantities; full count matrices, policy and transition tables with row SUM/Cons., and Tasks 2–4 printouts are in Appendix B (baseline block).

Table 3: Task 1 consistency checks (baseline, full dataset).

Source: consistency block under baseline Task 1 in Appendix B.

Check	Result
$\sum_k P(\alpha_k \mid \phi_i) = 1$ (visited ϕ_i)	PASS
$\sum_j P(\phi_j \mid \phi_i, \alpha_k) = 1$ (rows with mass)	PASS
Computed c_k matches empirical $\beta=1$ rate per action	PASS

Table 4: Baseline metrics on the full dataset ($n = 3518$ rows).

Quantity	Value
$ S , \bar{L}, \text{checksum } C$	80, 43.98, 83035
c_1, c_2, c_3	0.200, 0.382, 0.518
α^*, c_*	$\alpha_1, 0.200$
$\varepsilon, \min_i P(\alpha^* \mid \phi_i)$	0.370, 0.630 (state ϕ_5)
L_1 (actions / ϕ_{src} / ϕ_{dst})	0.016 / 0.015 / 0.016
Task 6 spread ($\alpha_1, \alpha_2, \alpha_3$)	0.051, 0.100, 0.244

Interpretation for baseline.

- **Is the system strongly optimal?** Weakly— ε is large: at least one visited state rarely chooses α^* under the empirical policy.
- **Is it noisy?** Yes—high ε and/or high spread across states suggests heterogeneous or stochastic-looking action choices.
- **Is behavior consistent across states?** No—behavior differs markedly across visited states (large spread in $P(\alpha^* \mid \phi)$).

Diagnostics: spread = 0.201, std $P(\alpha^* \mid \phi) = 0.077$ on visited states.

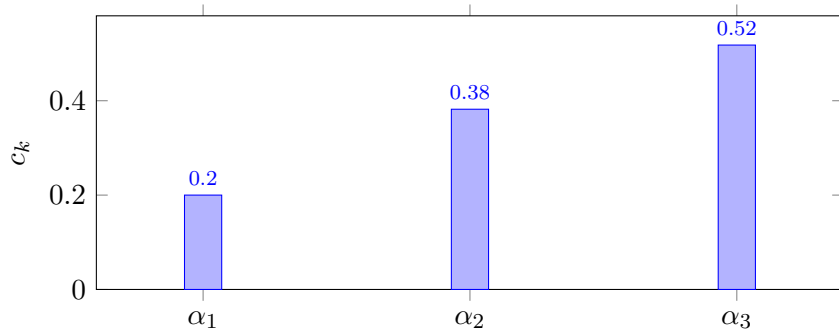


Figure 2: Global penalty probabilities c_k (baseline).

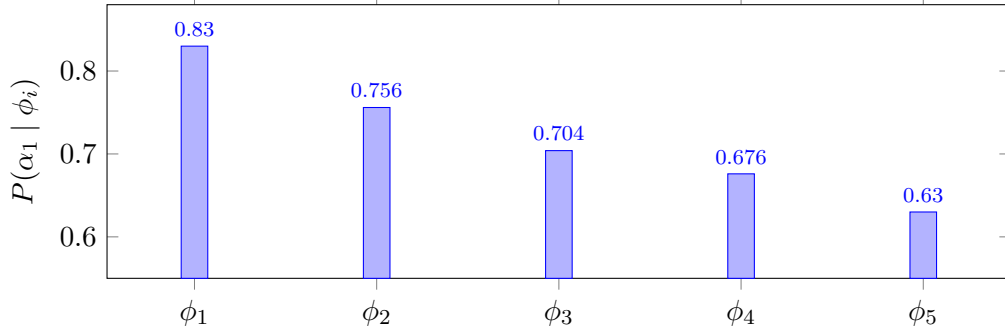


Figure 3: Empirical $P(\alpha^* | \phi_i)$ on visited states (baseline, $\alpha^* = \alpha_1$). State ϕ_5 attains the minimum and sets ε .

Figure 2 gives the global penalty ordering $c_1 < c_2 < c_3$ (0.20, 0.38, 0.52), so $\alpha^* = \alpha_1$ with $c_* \approx 0.20$. Figure 3 shows $P(\alpha_1 | \phi_i)$ decreasing from ϕ_1 to ϕ_5 (about 0.83 down to 0.63), so behavior is weakly ε -optimal despite uniform favoritism of α_1 .

4.2.2 Perturbation (20% row removal, 10 trials)

$\alpha^* = \alpha_1$ in *all* ten subsamples ($n = 2814$ rows each after removing 704 rows). Perturbed ε averages 0.371 ± 0.014 (mean shift +0.0013 vs. baseline 0.370); individual trials span $[0.351, 0.394]$. Action-marginal simulation error remains small ($L_1^{\text{act}} \in [0.0028, 0.0210]$, baseline 0.016). Task 6 spreads grow slightly under subsampling (e.g. α_2 : baseline spread 0.100 vs. perturbed mean 0.121), and the largest per-cell shift in $c_k(\phi_i)$ vs. baseline reaches 0.094 (mean 0.052 over trials). Tables 5 and 6 illustrate the first perturbation trial (trial 0, 704 rows removed, $n = 2814$); Table 7 aggregates all ten trials. The full perturbation log (Tasks 1–6 for each trial) is in Appendix B.

Table 5: Task 1 consistency checks (perturbation trial 0, $n = 2814$ rows).

Source: consistency block under **trial 0** Task 1 in Appendix B.

Check	Result
$\sum_k P(\alpha_k \phi_i) = 1$ (visited ϕ_i)	PASS
$\sum_j P(\phi_j \phi_i, \alpha_k) = 1$ (rows with mass)	PASS
Computed c_k matches empirical $\beta=1$ rate per action	PASS

Table 6: Perturbation trial 0 metrics ($n = 2814$ rows, 704 removed).

Quantity	Value
c_1, c_2, c_3	0.202, 0.389, 0.520
α^*, c_*	$\alpha_1, 0.202$
$\varepsilon, \min_i P(\alpha^* \phi_i)$	0.392, 0.608 (state ϕ_5)
L_1 (actions / ϕ_{src} / ϕ_{dst})	0.006 / 0.050 / 0.051
$ \bar{\beta}_{\text{data}} - \bar{\beta}_{\text{sim}} $ (global)	0.0003

Interpretation for perturbation (trial 0). The following answers refer to trial 0 only ($n = 2814$ rows after removing 704). Tasks 1–6 results for trials 1–9 are reported in Appendix B.

- **Is the system strongly optimal?** Weakly— ε is large: at least one visited state rarely chooses α^* under the empirical policy.

- **Is it noisy?** Yes—high ε and/or high spread across states suggests heterogeneous or stochastic-looking action choices.
- **Is behavior consistent across states?** No—behavior differs markedly across visited states (large spread in $P(\alpha^* | \phi)$).

Diagnostics: spread = 0.214, std $P(\alpha^* | \phi)$ = 0.082 on visited states (ε = 0.392 on the subsample).

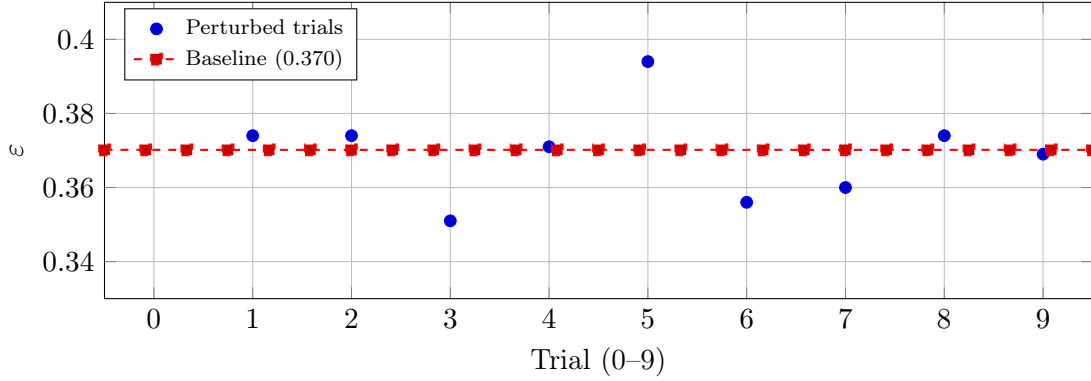


Figure 4: ε across perturbation trials vs. baseline (Task 3).

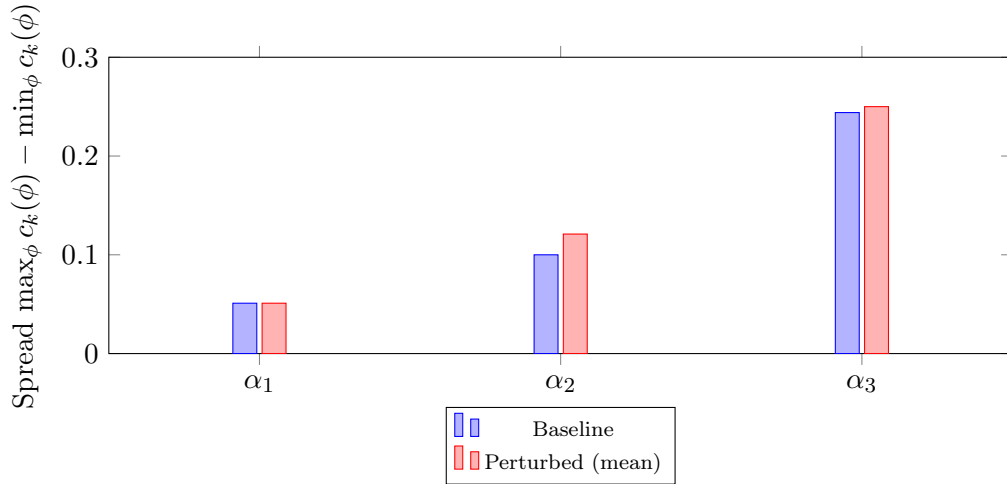


Figure 5: Task 6 spread of $c_k(\phi_i)$ across states: baseline vs. mean over perturbed trials.

Figure 4 plots ε for ten perturbation trials against the baseline dashed line at 0.370. Figure 5 compares Task 6 spreads; α_3 has the largest baseline spread (0.244), and perturbed means rise for α_2 and α_3 while α_1 stays near 0.051. Table 7 summarizes stability of α^* , ε , L_1^{act} , and per-state penalty drift.

4.3 Discussion

Taken together, the figures and tables describe a VSA that is easy to summarize globally but not uniformly optimal in state space. A single low-penalty action (Figure 2) coexists with substantial slippage in $P(\alpha^* | \phi_i)$ (Figure 3), so reporting only c_* would overstate how closely the controller follows α^* . The perturbation panel (Figure 4) and summary table show that this picture is stable under random row loss on the present sample, while the spread chart (Figure 5) warns against reading a strict action-only penalty law into the data. Simulation agreement in Table 4

Table 7: Perturbation summary (10 trials, 704 rows removed each).

Metric	Result
α^* (all trials)	α_1
ε : mean $\pm$ std (vs. baseline 0.370)	0.371 ± 0.014
L_1^{act} range (baseline 0.016)	[0.0028, 0.0210]
Task 6 spread mean ($\alpha_1, \alpha_2, \alpha_3$)	0.051, 0.121, 0.250
$ \Delta c_k(\phi_i) $ max / mean over trials	0.094 / 0.052

supports internal coherence of the estimated model as a descriptive fit, not as proof of a true generative process.

The analysis is limited to one log, non-smoothed counts, and i.i.d. row deletion that ignores episode structure; rare (ϕ_i, α_k) cells may still drive part of the spread in $c_k(\phi_i)$. Global c_k remain useful for ranking actions and match per-state tables on α^* , but $c_k(\phi_i)$ should be reported whenever a P-model is discussed. Extensions on other datasets, episode-level perturbations, and explicit state-dependent penalty models would test how far these patterns generalize.

5 Conclusion

This paper presented count-based empirical identification of a variable-structure automaton under the P-model on a fixed interaction dataset (3518 steps, 80 episodes). A single reproducible pipeline validated the log, estimated policies and penalties, assessed ε -optimality, checked a simulator, stressed the fit under random row removal, and audited per-state penalties against the global P-model.

The principal outcomes are that α_1 is the lowest-penalty action with weak ε -optimality overall, simulated marginals track the data closely, and global estimates are stable under the chosen subsampling experiment, while per-state penalties show measurable spread that qualifies a strict action-only penalty law. Interpretation, limitations, and extensions are developed in Section 4.3.

APPENDIX

Appendix A: Supplementary source code listing

Excerpts below are taken from the submitted source files, one subsection per module. Line ranges match the repository at report time; together they implement the flow in Section 3.1 (Algorithm 1) and the estimators in Section 2.2.

A.1 fingerprint.py

```
df = load_vsa_dataset(csv_path)

n_rows = len(df)
n_sequences = int(df[CLMN_SEQ_ID].nunique())
lengths = df.groupby(CLMN_SEQ_ID, sort=False).size()
mean_sequence_length = float(lengths.mean())

penalty_rate_per_action = (
    df.groupby(CLMN_ALPHA)[CLMN_BETA].mean().sort_index()
)
penalty_rate_per_action.index.name = CLMN_ALPHA
penalty_rate_per_action.name = "empirical_penalty_rate"

checksum = int(df[CLMN_STEP].sum())
```

A.2 estimation.py

_format_sum_cons_table and _format_penalty_table:

```
def _format_sum_cons_table(
    prob_df: pd.DataFrame,
    active_mask: pd.Series,
    *,
    atol: float = 1e-9,
) -> str:
    """Probability table with row sums and per-row PASS/FAIL consistency."""
    display = prob_df.round(6).copy()
    row_sums = prob_df.sum(axis=1)
    sum_col: list[object] = []
    cons_col: list[str] = []
    for idx in prob_df.index:
        s = float(row_sums.loc[idx])
        if not bool(active_mask.loc[idx]):
            sum_col.append(round(s, 6) if s != 0.0 else 0)
            cons_col.append("-")
        elif np.isclose(s, 1.0, atol=atol):
            sum_col.append(1)
            cons_col.append("PASS")
        else:
            sum_col.append(round(s, 6))
            cons_col.append("FAIL")
    display["SUM"] = sum_col
    display["Cons."] = cons_col
    return display.to_string()

_PENALTY_CONS_PASS = "c_k matches empirical frequencies"
_PENALTY_CONS_FAIL = "c_k does not match empirical frequencies"
```

```

def _format_penalty_table(
    penalty: pd.Series,
    empirical_penalty: pd.Series,
    *,
    atol: float = 1e-9,
) -> str:
    """Penalty table with stacked column headers and per-action consistency text."""
    index_label = "alpha"
    val_w = 12
    rows: list[tuple[str, float, float, str]] = []
    for a in penalty.index:
        ck = float(penalty.loc[a])
        emp = float(empirical_penalty.loc[a])
        ok = np.isclose(ck, emp, atol=atol)
        rows.append(
            (
                str(a),
                ck,
                emp,
                _PENALTY_CONS_PASS if ok else _PENALTY_CONS_FAIL,
            )
        )

    idx_w = max(len(index_label), *(len(r[0]) for r in rows))
    lines = [
        f"{'':>{idx_w}} {'c_k':>{val_w}} {'empirical':>{val_w}} Cons.",
        f"{index_label:>{idx_w}} {'':>{val_w}} {'penalty':>{val_w}}",
    ]
    for name, ck, emp, cons in rows:
        lines.append(
            f"{name:>{idx_w}} {ck:>{val_w}.6f} {emp:>{val_w}.6f} {cons}"
        )
    return "\n".join(lines)

```

run_estimation (probabilities, consistency checks, result):

```

row_totals_phi = N_phi_alpha.sum(axis=1)
policy = N_phi_alpha.div(row_totals_phi.replace(0, np.nan), axis=0).fillna(0.0)

transition: Dict[str, pd.DataFrame] = {}
for a in actions:
    N = N_phi_alpha_phi[a]
    T = N.div(N.sum(axis=1).replace(0, np.nan), axis=0).fillna(0.0)
    transition[a] = T

total_per_action = N_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty = (N_alpha_beta[1] / total_per_action).fillna(0.0)
penalty.name = "c_k"

N_phi_alpha_beta = (
    df.groupby([CLMN_PHI_SRC, CLMN_ALPHA, CLMN_BETA]).size()
    .unstack(fill_value=0)
    .reindex(columns=betas, fill_value=0)
)
totals_phi_alpha = N_phi_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty_per_state = (
    (N_phi_alpha_beta[1] / totals_phi_alpha)
    .fillna(0.0)
    .unstack(CLMN_ALPHA)
    .reindex(index=states, columns=actions, fill_value=0.0)
)
penalty_per_state.index.name = CLMN_PHI_SRC

```

```

penalty_per_state.columns.name = CLMN_ALPHA

empirical_penalty = (
    df.groupby(CLMN_ALPHA)[CLMN_BETA].mean().reindex(actions, fill_value=0.0)
)
empirical_penalty.name = "empirical_penalty"

# Consistency checks -----
nonzero_phi = row_totals_phi > 0
policy_rows_ok = bool(
    np.allclose(policy.loc[nonzero_phi].sum(axis=1).values, 1.0, atol=1e-9)
)

transition_rows_ok = True
for a, T in transition.items():
    N = N_phi_alpha_phi[a]
    nonzero = N.sum(axis=1) > 0
    if not np.allclose(T.loc[nonzero].sum(axis=1).values, 1.0, atol=1e-9):
        transition_rows_ok = False
        break

penalty_matches_empirical = bool(
    np.allclose(penalty.values, empirical_penalty.values, atol=1e-9)
)

checks = {
    "sum_k P(alpha_k | phi_i) == 1": policy_rows_ok,
    "sum_j P(phi_j | phi_i, alpha_k) == 1": transition_rows_ok,
    "c_k matches empirical frequencies": penalty_matches_empirical,
}

result = EstimationResult(
    counts_phi_alpha=N_phi_alpha,
    counts_phi_alpha_phi=N_phi_alpha_phi,
    counts_alpha_beta=N_alpha_beta,
    policy=policy,
    transition=transition,
    penalty=penalty,
    penalty_per_state=penalty_per_state,
    empirical_penalty=empirical_penalty,
    checks=checks,
    states=states,
    actions=actions,
    n_rows=len(df),
)

```

Report printing (policy, transitions, penalties with SUM/Cons.):

```

lines.append(_section("Policy P(alpha_k | phi_i)"))
lines.append(_format_sum_cons_table(policy, nonzero_phi))

for a in actions:
    lines.append(_section(f"Transition P(phi_j | phi_i, alpha_k={a})"))
    transition_active = N_phi_alpha_phi[a].sum(axis=1) > 0
    lines.append(_format_sum_cons_table(transition[a], transition_active))

lines.append(_section("Penalty probabilities c_k = P(beta=1 | alpha_k)"))
side_by_side = pd.concat([penalty, empirical_penalty], axis=1).round(6)
lines.append(_format_penalty_table(penalty, empirical_penalty))

if print_penalty_per_state:
    lines.append(_section("Task 6 --Model assumption check"))
    lines.append("c_k(phi_i) = P(beta=1 | alpha_k, phi_i)")
    lines.append(_fmt(penalty_per_state.round(6)))

```

```

lines.append(_section("Consistency checks"))
for name, ok in checks.items():
    lines.append(f" [{ 'PASS' if ok else 'FAIL' }] {name}")

```

A.3 optimal_action_identification.py

```

c_min = float(penalty.min())
ties: List[str] = [str(a) for a in penalty.index[np.isclose(penalty.values, c_min)]]
alpha_star = ties[0] # deterministic pick: first action tied at the minimum

ranking = (
    penalty.sort_values(ascending=True)
        .to_frame()
        .assign(rank=lambda d: np.arange(1, len(d) + 1, dtype=int))
)
ranking["is_optimal"] = np.isclose(ranking["c_k"].values, c_min)
ranking.index.name = CLMN_ALPHA

result = OptimalActionResult(
    alpha_star=alpha_star,
    c_star=c_min,
    ties=ties,
    ranking=ranking,
    penalty=penalty,
    source=source,
)

```

A.4 e_optimality_analysis.py

```

def _compute_core(
    policy: pd.DataFrame,
    counts_phi_alpha: pd.DataFrame,
    alpha_star: str,
) -> Tuple[float, float, pd.Series, List[str]]:
    if alpha_star not in policy.columns:
        raise ValueError(
            f"alpha* column {alpha_star!r} not in policy columns {list(policy.columns)}. "
            "Regenerate policy.csv / run Task 1 after Task 2."
        )
    row_totals = counts_phi_alpha.sum(axis=1)
    visited = row_totals > 0
    if not bool(visited.any()):
        raise ValueError("No visited states in count matrix; cannot compute epsilon.")

    p_col = policy.loc[visited, alpha_star].astype(float)
    min_p = float(np.min(p_col.values))
    epsilon = 1.0 - min_p
    tol = 1e-9
    argmin_states = [str(s) for s in p_col.index[np.isclose(p_col.values, min_p, atol=tol)]]
    return epsilon, min_p, p_col, argmin_states

```

A.5 simulation.py

```

def _simulate_block(
    real_df: pd.DataFrame,
    policy: pd.DataFrame,
    transition: Dict[str, pd.DataFrame],

```



```

penalty: pd.Series,
rng: np.random.Generator,
) -> pd.DataFrame:
    """Generate synthetic rows matching real (sequence_id, step) layout."""
    meta = (
        real_df.sort_values([CLMN_SEQ_ID, CLMN_STEP])
        .groupby(CLMN_SEQ_ID, sort=False)
        .agg(length=(CLMN_STEP, "count"), phi0=(CLMN_PHI_SRC, "first"))
    )
    lengths = meta["length"].astype(int).to_numpy()
    phi0s = meta["phi0"].astype(str).to_numpy()
    seq_ids = meta.index.to_numpy()

    states = list(policy.index.astype(str))
    actions = list(policy.columns.astype(str))

    rows: List[dict] = []
    for sid, L, phi in zip(seq_ids, lengths, phi0s):
        phi = str(phi)
        if phi not in policy.index:
            phi = states[0]
        for step in range(1, int(L) + 1):
            row = policy.loc[phi].to_numpy(dtype=float)
            a = _sample_from_row(rng, actions, row)
            tmat = transition[a]
            labels = list(tmat.columns.astype(str))
            if phi in tmat.index:
                row = tmat.loc[phi].to_numpy(dtype=float)
            else:
                row = np.ones(len(labels), dtype=float) / len(labels)
            phi_next = _sample_from_row(rng, labels, row)
            p_pen = float(penalty.reindex([a]).fillna(0.0).iloc[0])
            p_pen = min(max(p_pen, 0.0), 1.0)
            beta = int(rng.binomial(1, p_pen))
            rows.append(
                {
                    CLMN_SEQ_ID: int(sid),
                    CLMN_STEP: step,
                    CLMN_PHI_SRC: phi,
                    CLMN_PHI_DST: phi_next,
                    CLMN_ALPHA: a,
                    CLMN_BETA: beta,
                }
            )
            phi = phi_next

    return pd.DataFrame(rows)

```

A.6 main.py

`_drop_rows_randomly:`

```

def _drop_rows_randomly(
    df: pd.DataFrame,
    fraction_remove: float,
    rng: np.random.Generator,
) -> Tuple[pd.DataFrame, int]:
    """Return a copy of 'df' with 'round(fraction_remove * n)' rows removed."""
    n = len(df)
    if n <= 1 or fraction_remove <= 0.0:
        return df.copy(), 0
    n_remove = int(round(fraction_remove * n))

```

```

n_remove = min(max(0, n_remove), n - 1)
if n_remove <= 0:
    return df.copy(), 0
drop_positions = rng.choice(n, size=n_remove, replace=False)
out = df.drop(df.index[drop_positions]).reset_index(drop=True)
return out, n_remove

```

`_run_tasks_1_to_4:`

```

def _run_tasks_1_to_4(
    df: pd.DataFrame,
    csv_path: Path,
    *,
    dataset_label: str,
    print_to_console: bool,
) -> PipelineRunResult:
    estimation = run_estimation(
        df,
        dataset_label=dataset_label,
        output_dir=None,
        print_to_console=print_to_console,
        print_penalty_per_state=False,
    )
    optimal = run_optimal_action(
        estimation=estimation,
        output_dir=None,
        print_to_console=print_to_console,
    )
    epsilon_res = run_epsilon_optimality(
        estimation=estimation,
        optimal=optimal,
        output_dir=None,
        print_to_console=print_to_console,
    )
    sim_res = run_simulation(
        estimation=estimation,
        csv_path=csv_path,
        output_dir=None,
        print_to_console=print_to_console,
    )
    return PipelineRunResult(
        label=dataset_label,
        n_rows=len(df),
        alpha_star=optimal.alpha_star,
        c_star=optimal.c_star,
        c_k_per_state=estimation.penalty_per_state.copy(),
        epsilon=epsilon_res.epsilon,
        min_p_alpha_star=epsilon_res.min_p_alpha_star,
        l1_action=sim_res.l1_action,
        l1_phi_src=sim_res.l1_phi_src,
        l1_phi_dst=sim_res.l1_phi_dst,
    )

```

`_print_c_k_per_state_runs:`

```

def _print_c_k_per_state_runs(runs: List[PipelineRunResult]) -> None:
    """Print per-state penalty tables for one or more pipeline runs."""
    lines = [
        _section("Task 6 --Model assumption check"),
        "c_k(phi_i) = P(beta=1 | alpha_k, phi_i)",
        "",
    ]
    for run in runs:
        lines.extend(_format_c_k_per_state_lines(run))

```

```
print("\n".join(lines))
```

_print_experiments_summary (spread and table-delta helpers):

```
def _spread_across_states(c_k_per_state: pd.DataFrame) -> pd.Series:
    """Per action: max_phi c_k(phi) - min_phi c_k(phi) (P-model spread diagnostic)."""
    return c_k_per_state.max(axis=0) - c_k_per_state.min(axis=0)

def _max_abs_table_delta(reference: pd.DataFrame, other: pd.DataFrame) -> float:
    """Largest |c_k(phi, alpha) - reference| over aligned cells."""
    aligned = other.reindex_like(reference).fillna(0.0)
    return float(np.abs(aligned - reference).max().max())

def _print_experiments_summary(
    *,
    csv_path: Path,
    n_full: int,
    perturbation_rate: float,
    experiment_count: int,
    baseline: PipelineRunResult,
    perturbed: List[PipelineRunResult],
) -> None:
    epsilons = np.array([r.epsilon for r in perturbed], dtype=float)
    baseline_spread = _spread_across_states(baseline.c_k_per_state)
    perturbed_spreads = np.array(
        [_spread_across_states(r.c_k_per_state).values for r in perturbed],
        dtype=float,
    )
    table_deltas = np.array(
        [_max_abs_table_delta(baseline.c_k_per_state, r.c_k_per_state) for r in perturbed],
        dtype=float,
    )
    actions = list(baseline.c_k_per_state.columns)

    lines = [
        _section(
            f"Task 5 --Perturbation "
            f"({perturbation_rate:.0f}% rows removed, {experiment_count} trial(s))"
        ),
        f"Dataset : {csv_path.resolve()}",
        f"Rows : {n_full} full",
        "",
        "Baseline (full data, Tasks 1--4)",
        f" alpha* = {baseline.alpha_star} c* = {baseline.c_star:.6f}",
        f" epsilon = {baseline.epsilon:.6f}",
        "",
        "Perturbed runs",
    ]
    for r in perturbed:
        lines.append(
            f" {r.label}: alpha*={r.alpha_star} epsilon={r.epsilon:.6f} "
            f"L1_act={r.l1_action:.4f}"
        )
    if len(perturbed) > 1:
        lines.extend(
            [
                "",
                f" epsilon mean={epsilons.mean():.6f} std={epsilons.std(ddof=1):.6f} "
                f"delta_mean_vs_baseline={epsilons.mean() - baseline.epsilon:+.6f}",
            ]
        )
    elif len(perturbed) == 1:
```

```

        lines.append(
            f"\n delta_epsilon (perturbed - baseline) = "
            f"{perturbed[0].epsilon - baseline.epsilon:+.6f}"
        )

    lines.extend(
        [
            "Spread across states per action (max_phi c_k - min_phi c_k):",
            " Baseline: "
            + " ".join(f"{a}={baseline_spread[a]:.6f}" for a in actions),
        ]
    )

    if perturbed:
        if len(perturbed) > 1:
            spread_mean = perturbed_spreads.mean(axis=0)
            spread_std = perturbed_spreads.std(axis=0, ddof=1)
            lines.append(
                " Perturbed: "
                + " ".join(
                    f"{a}=mean {spread_mean[i]:.6f} std {spread_std[i]:.6f}"
                    for i, a in enumerate(actions)
                )
            )
            lines.extend(
                [
                    "",
                    " |c_k_per_state - baseline| max cell delta:",
                    f" mean={table_deltas.mean():.6f} "
                    f"std={table_deltas.std(ddof=1):.6f} "
                    f"max={table_deltas.max():.6f}",
                ]
            )
        else:
            p_spread = _spread_across_states(perturbed[0].c_k_per_state)
            lines.append(
                " Perturbed: "
                + " ".join(f"{a}={p_spread[a]:.6f}" for a in actions)
            )
            lines.append(
                f"\n |c_k_per_state - baseline| max cell delta: "
                f"{table_deltas[0]:.6f}"
            )

    print("\n".join(lines))
    _print_c_k_per_state_runs([baseline, *perturbed])

```

run_all:

```

def run_all(
    csv_path: Union[Path, str],
    *,
    perturbation_rate: float,
    experiment_count: int = 1,
    random_seed: int,
    print_to_console: bool = True,
) -> List[PipelineRunResult]:
    """Fingerprint, baseline Tasks 1--4 on full data, then optional perturbed passes.

    Parameters
    -----
    csv_path
        Path to the VSA interaction CSV.
    perturbation_rate
        Percent of rows to remove at random before each perturbed pass (0--99).

```

```

    ‘‘0‘‘: baseline only. ‘‘20‘‘: baseline plus ‘‘experiment_count‘‘ trials
    with 20% of rows removed.
experiment_count
    Number of perturbed passes when ‘‘perturbation_rate > 0‘‘.
random_seed
    Base RNG seed for row removal.
print_to_console
    Whether to print fingerprint, per-task reports (baseline only when rate is 0),
    and the final summary.

Returns
-----
list of PipelineRunResult
    ‘‘[baseline, ...perturbed_trials]‘‘.
"""
if perturbation_rate < 0 or perturbation_rate >= 100:
    raise ValueError("perturbation_rate must be in [0, 100).")
if experiment_count < 1:
    raise ValueError("experiment_count must be >= 1.")
if perturbation_rate == 0 and experiment_count != 1:
    warnings.warn(
        "experiment_count is ignored when perturbation_rate is 0.",
        stacklevel=2,
    )

csv_path = Path(csv_path)
df_full = load_vsa_dataset(csv_path)

run_fingerprint(
    csv_path,
    output_dir=None,
    print_to_console=print_to_console,
)

# first run without perturbation
baseline = _run_tasks_1_to_4(
    df_full,
    csv_path,
    dataset_label=f"{csv_path.name} (baseline)",
    print_to_console=print_to_console,
)
results: List[PipelineRunResult] = [baseline]
if print_to_console:
    _print_run_results(baseline)

if perturbation_rate > 0:
    fraction = perturbation_rate / 100.0
    rng_master = np.random.default_rng(random_seed)
    for t in range(experiment_count):
        trial_seed = int(rng_master.integers(0, 2**31 - 1))
        rng = np.random.default_rng(trial_seed)
        df_p, n_removed = _drop_rows_randomly(df_full, fraction, rng)
        perturbation_run = _run_tasks_1_to_4(
            df_p,
            csv_path,
            dataset_label=f"{csv_path.name} (trial {t}, removed {n_removed})",
            print_to_console=print_to_console,
        )
        if print_to_console:
            _print_run_results(perturbation_run)
        results.append(perturbation_run)

if print_to_console:

```

```

        _print_experiments_summary(
            csv_path=csv_path,
            n_full=len(df_full),
            perturbation_rate=perturbation_rate,
            experiment_count=experiment_count,
            baseline=baseline,
            perturbed=results[1:],
        )
    elif print_to_console:
        _print_c_k_per_state_runs([baseline])

    return results

```

A.7 util.py

Column constants, shared result types, and CSV loading:

```

# DataFrame / CSV column names (headerless file column order matches COLUMNS)
CLMN_SEQ_ID = "sequence_id"
CLMN_STEP = "step"
CLMN_PHI_SRC = "phi_src"
CLMN_PHI_DST = "phi_dst"
CLMN_ALPHA = "alpha"
CLMN_BETA = "beta"

COLUMNS = [
    CLMN_SEQ_ID,
    CLMN_STEP,
    CLMN_PHI_SRC,
    CLMN_PHI_DST,
    CLMN_ALPHA,
    CLMN_BETA,
]

@dataclass
class EstimationResult:
    """Structured return value of 'estimation.run_estimation'."""

    counts_phi_alpha: pd.DataFrame # N(phi_i, alpha_k)
    counts_phi_alpha_phi: Dict[str, pd.DataFrame] # N(phi_i, alpha_k, phi_j) per alpha
    counts_alpha_beta: pd.DataFrame # N(alpha_k, beta)
    policy: pd.DataFrame # P(alpha_k | phi_i)
    transition: Dict[str, pd.DataFrame] # P(phi_j | phi_i, alpha_k) per alpha
    penalty: pd.Series # c_k = P(beta=1 | alpha_k)
    penalty_per_state: pd.DataFrame # c_k(phi_i) = P(beta=1 | alpha_k, phi_i)
    empirical_penalty: pd.Series # groupby(CLMN_ALPHA)[CLMN_BETA].mean()
    checks: Dict[str, bool] # named pass/fail flags
    states: list
    actions: list
    n_rows: int

```

```

@dataclass
class OptimalActionResult:
    """Structured return value of 'optimal_action_identification.run_optimal_action'."""

```

```

    c_star: float # its penalty probability (the minimum c_k)
    ties: List[str] # all actions tied at the minimum (usually just [alpha_star])
    ranking: pd.DataFrame # actions sorted by c_k (ascending), columns: c_k, rank
    penalty: pd.Series # the c_k vector echoed back (action -> c_k)
    source: str # "estimation-result" or absolute path to penalty.csv

```

```
def load_vsa_dataset(csv_path: Union[Path, str]) -> pd.DataFrame:
    """Load the headerless UTF-8 VSA interaction CSV.

    Each row is
    '(CLMN_SEQ_ID, CLMN_STEP, CLMN_PHI_SRC, CLMN_PHI_DST, CLMN_ALPHA, CLMN_BETA)'
    with integer 'CLMN_SEQ_ID', 'CLMN_STEP', and 'CLMN_BETA'; state and
    action columns stay as strings (e.g. Greek labels).
    """
    csv_path = Path(csv_path)
    df = pd.read_csv(csv_path, header=None, names=COLUMNS, encoding="utf-8")
    df[CLMN_SEQ_ID] = df[CLMN_SEQ_ID].astype(int)
    df[CLMN_STEP] = df[CLMN_STEP].astype(int)
    df[CLMN_BETA] = df[CLMN_BETA].astype(int)
    return df
```

Appendix B: Full experimental log

Complete console output from python main.py (load_all: perturbation_rate=20, ten trials, random_seed=42) on Engin-vsa_dataset_full.csv. Task 1 tables include row SUM and Cons. columns for policies and transitions, and per-action penalty consistency messages alongside c_k and empirical penalty rates. Greek labels are transliterated for typesetting ($a_1 \equiv \alpha_1$, $p_1 \equiv \phi_1$); the raw UTF-8 log is experiments.txt (2553 lines).

```
D:\Documents\00-PhD\04-CSE8003\assignments\term_project\proj_python\.venv\Scripts\python.exe D:\Documents
\00-PhD\04-CSE8003\assignments\term_project\proj_python\main.py

=====
Dataset fingerprint (Engin-vsa_dataset_full.csv)
=====
CSV path : D:\Documents\00-PhD\04-CSE8003\assignments\term_project\Engin-vsa_dataset_full.csv
Total number of rows : 3518
Number of sequences : 80
Mean sequence length : 43.975000

Empirical penalty rate per action (mean of beta given alpha):
alpha
a1 0.199608
a2 0.381579
a3 0.517606

Checksum (sum of step over all rows):
checksum = 83035

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (baseline))
=====
Rows in dataset : 3518
States observed : ['p1', 'p2', 'p3', 'p4', 'p5']
Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====
alpha a1 a2 a3
phi_src
p1 573 78 39
p2 665 157 58
p3 511 157 58
p4 459 154 66
p5 342 138 63

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
```

```

p1 225 228 40 37 43
p2 198 197 192 45 33
p3 36 166 134 140 35
p4 28 40 131 128 132
p5 29 33 22 131 127

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 27 32 8 9 2
p2 43 54 42 9 9
p3 10 52 45 43 7
p4 14 18 43 32 47
p5 15 13 10 51 49

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 16 12 3 4 4
p2 20 15 18 2 3
p3 4 16 21 14 3
p4 5 4 20 17 20
p5 8 7 4 24 20

=====
N(alpha_k, beta) -- raw counts
=====
beta 0 1
alpha
a1 2041 509
a2 423 261
a3 137 147

=====
Policy P(alpha_k | phi_i)
=====
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.830435 0.113043 0.056522 1 PASS
p2 0.755682 0.178409 0.065909 1 PASS
p3 0.703857 0.216253 0.079890 1 PASS
p4 0.675994 0.226804 0.097202 1 PASS
p5 0.629834 0.254144 0.116022 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.392670 0.397906 0.069808 0.064572 0.075044 1 PASS
p2 0.297744 0.296241 0.288722 0.067669 0.049624 1 PASS
p3 0.070450 0.324853 0.262231 0.273973 0.068493 1 PASS
p4 0.061002 0.087146 0.285403 0.278867 0.287582 1 PASS
p5 0.084795 0.096491 0.064327 0.383041 0.371345 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.346154 0.410256 0.102564 0.115385 0.025641 1 PASS
p2 0.273885 0.343949 0.267516 0.057325 0.057325 1 PASS
p3 0.063694 0.331210 0.286624 0.273885 0.044586 1 PASS
p4 0.090909 0.116883 0.279221 0.207792 0.305195 1 PASS
p5 0.108696 0.094203 0.072464 0.369565 0.355072 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

```



```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
118 p1 0.410256 0.307692 0.076923 0.102564 0.102564 1 PASS
p2 0.344828 0.258621 0.310345 0.034483 0.051724 1 PASS
p3 0.068966 0.275862 0.362069 0.241379 0.051724 1 PASS
p4 0.075758 0.060606 0.303030 0.257576 0.303030 1 PASS
p5 0.126984 0.111111 0.063492 0.380952 0.317460 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.199608 0.199608 c_k matches empirical frequencies
a2 0.381579 0.381579 c_k matches empirical frequencies
a3 0.517606 0.517606 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.199608
a2 0.381579
a3 0.517606

alpha* = a1
c_* = 0.199608

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.199608 1 True
a2 0.381579 2 False
a3 0.517606 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.629834
e = 1 - min_i P(a*|p_i) = 0.370166
Visited states (count): 5
168 Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.830435
p2 0.755682
p3 0.703857
p4 0.675994
p5 0.629834

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

Diagnostics: spread(max-min P)=0.200601, std(P)=0.077124

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.007391

a2 0.007959

a3 0.000569

L1 sum of abs diffs (actions): 0.015918

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.003127

p2 0.004548

p3 0.001990

p4 0.001137

p5 0.004548

L1 sum: 0.015350

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.004548

p2 0.003695

p3 0.003127

p4 0.000569

p5 0.004548

L1 sum: 0.016487

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.012496

a2 0.005616

a3 0.020856

L1 sum (per-action |D|): 0.038968

Global penalty rate |D|: data=0.260659 sim=0.252985 |D|=0.007675

=====
main: Engin-vsa_dataset_full.csv (baseline) - Tasks 1--4 finished
=====

rows processed : 3518

optimal action : alpha* = a1 (c* = 0.199608)

epsilon (Task 3) : e = 0.370166 (min P(a*|p) = 0.629834)

simulation L1 : actions=0.0159 phi_src=0.0153 phi_dst=0.0165

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 0, removed 704))
=====

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====

alpha a1 a2 a3

phi_src

p1 453 67 31

p2 529 128 50

p3 401 137 45

p4 360 128 51

p5 264 118 52

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5

phi_src

268

```
p1 171 188 28 32 34
p2 166 155 149 34 25
p3 24 136 109 102 30
p4 22 37 105 98 98
p5 25 26 16 100 97
```

```
=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 25 27 6 7 2
p2 35 49 32 8 4
p3 9 48 40 34 6
p4 11 14 35 29 39
p5 10 10 8 45 45
```

```
=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 14 9 2 3 3
p2 16 12 17 2 3
p3 1 15 15 12 2
p4 5 3 16 14 13
p5 7 6 4 19 16
```

```
=====
N(alpha_k, beta) -- raw counts
=====
```

```
beta 0 1
alpha
a1 1601 406
a2 353 225
a3 110 119
```

```
=====
Policy P(alpha_k | phi_i)
=====
```

```
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.822142 0.121597 0.056261 1 PASS
p2 0.748232 0.181047 0.070721 1 PASS
p3 0.687822 0.234991 0.077187 1 PASS
p4 0.667904 0.237477 0.094620 1 PASS
p5 0.608295 0.271889 0.119816 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.377483 0.415011 0.061810 0.070640 0.075055 1 PASS
p2 0.313800 0.293006 0.281664 0.064272 0.047259 1 PASS
p3 0.059850 0.339152 0.271820 0.254364 0.074813 1 PASS
p4 0.061111 0.102778 0.291667 0.272222 0.272222 1 PASS
p5 0.094697 0.098485 0.060606 0.378788 0.367424 1 PASS
```

318

```
=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.373134 0.402985 0.089552 0.104478 0.029851 1 PASS
p2 0.273438 0.382812 0.250000 0.062500 0.031250 1 PASS
p3 0.065693 0.350365 0.291971 0.248175 0.043796 1 PASS
p4 0.085938 0.109375 0.273438 0.226562 0.304688 1 PASS
p5 0.084746 0.084746 0.067797 0.381356 0.381356 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====
```

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.451613 0.290323 0.064516 0.096774 0.096774 1 PASS
p2 0.320000 0.240000 0.340000 0.040000 0.060000 1 PASS
p3 0.022222 0.333333 0.333333 0.266667 0.044444 1 PASS
p4 0.098039 0.058824 0.313725 0.274510 0.254902 1 PASS
p5 0.134615 0.115385 0.076923 0.365385 0.307692 1 PASS

```

```

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====

```

```

c_k empirical Cons.
alpha penalty
a1 0.202292 0.202292 c_k matches empirical frequencies
a2 0.389273 0.389273 c_k matches empirical frequencies
a3 0.519651 0.519651 c_k matches empirical frequencies

```

```

=====
Consistency checks
=====

```

```

[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

```

```

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====

```

```

Penalty probabilities c_k = P(beta = 1 | alpha_k):

```

```

alpha
a1 0.202292
a2 0.389273
a3 0.519651

```

```

alpha* = a1
c_* = 0.202292

```

```

=====
Ranking by c_k (ascending)
=====

```

```

c_k rank is_optimal
alpha
a1 0.202292 1 True
a2 0.389273 2 False
a3 0.519651 3 False

```

```

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====

```

```

a* = a1
min_i P(a*|p_i) over visited states = 0.608295
e = 1 - min_i P(a*|p_i) = 0.391705
Visited states (count): 5
Argmin state(s) (ties): ['p5']

```

```

P(a*|p_i) on visited states:

```

```

phi_src
p1 0.822142
p2 0.748232
p3 0.687822
p4 0.667904
p5 0.608295

```

```

=====
Interpretation (assignment prompts)
=====

```

```

* Is the system strongly optimal?
  Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
  Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
  choices.
* Is behavior consistent across states?
  No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

368

Diagnostics: spread(max-min P)=0.213847, std(P)=0.081542

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.002843

a2 0.001137

a3 0.001706

L1 sum of abs diffs (actions): 0.005685

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.004832

p2 0.020182

p3 0.002843

p4 0.009949

p5 0.012223

L1 sum: 0.050028

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.006822

p2 0.018761

p3 0.003127

p4 0.010233

p5 0.012223

L1 sum: 0.051165

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.005513

a2 0.006502

a3 0.020325

L1 sum (per-action |D|): 0.032341

Global penalty rate |D|: data=0.260659 sim=0.260375 |D|=0.000284

=====
main: Engin-vsa_dataset_full.csv (trial 0, removed 704) - Tasks 1--4 finished
=====

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.202292)

epsilon (Task 3) : e = 0.391705 (min P(a*|p) = 0.608295)

simulation L1 : actions=0.0057 phi_src=0.0500 phi_dst=0.0512

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 1, removed 704))
=====

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====

alpha a1 a2 a3

phi_src

p1 448 62 29

p2 532 134 41

p3 415 126 45

p4 377 122 61

p5 264 108 50

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5

phi_src

p1 178 180 27 29 34
p2 159 162 150 34 27
p3 30 131 112 114 28
p4 21 36 110 105 105
p5 25 20 16 105 98

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 20 23 8 9 2
p2 37 44 38 9 6
p3 9 43 35 34 5
p4 12 16 33 24 37
p5 12 12 6 40 38

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 15 8 3 1 2
p2 12 10 16 1 2
p3 4 13 15 10 3
p4 5 4 19 13 20
p5 8 6 4 15 17

=====
N(alpha_k, beta) -- raw counts
=====

beta 0 1
alpha
a1 1631 405
a2 339 213
a3 107 119

=====
Policy P(alpha_k | phi_i)
=====

alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.831169 0.115028 0.053803 1 PASS
p2 0.752475 0.189533 0.057992 1 PASS
p3 0.708191 0.215017 0.076792 1 PASS
p4 0.673214 0.217857 0.108929 1 PASS
p5 0.625592 0.255924 0.118483 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.397321 0.401786 0.060268 0.064732 0.075893 1 PASS
p2 0.298872 0.304511 0.281955 0.063910 0.050752 1 PASS
p3 0.072289 0.315663 0.269880 0.274699 0.067470 1 PASS
p4 0.055703 0.095491 0.291777 0.278515 0.278515 1 PASS
p5 0.094697 0.075758 0.060606 0.397727 0.371212 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.322581 0.370968 0.129032 0.145161 0.032258 1 PASS
p2 0.276119 0.328358 0.283582 0.067164 0.044776 1 PASS
p3 0.071429 0.341270 0.277778 0.269841 0.039683 1 PASS
p4 0.098361 0.131148 0.270492 0.196721 0.303279 1 PASS
p5 0.111111 0.111111 0.055556 0.370370 0.351852 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

518

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.517241 0.275862 0.103448 0.034483 0.068966 1 PASS
p2 0.292683 0.243902 0.390244 0.024390 0.048780 1 PASS
p3 0.088889 0.288889 0.333333 0.222222 0.066667 1 PASS
p4 0.081967 0.065574 0.311475 0.213115 0.327869 1 PASS
p5 0.160000 0.120000 0.080000 0.300000 0.340000 1 PASS

```

```

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====

```

```

c_k empirical Cons.
alpha penalty
a1 0.198919 0.198919 c_k matches empirical frequencies
a2 0.385870 0.385870 c_k matches empirical frequencies
a3 0.526549 0.526549 c_k matches empirical frequencies

```

```

=====
Consistency checks
=====

```

```

[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

```

```

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====

```

```

Penalty probabilities c_k = P(beta = 1 | alpha_k):

```

```

alpha
a1 0.198919
a2 0.385870
a3 0.526549

```

```

alpha* = a1
c_* = 0.198919

```

```

=====
Ranking by c_k (ascending)
=====

```

```

c_k rank is_optimal
alpha
a1 0.198919 1 True
a2 0.385870 2 False
a3 0.526549 3 False

```

```

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====

```

```

a* = a1
min_i P(a*|p_i) over visited states = 0.625592
e = 1 - min_i P(a*|p_i) = 0.374408
Visited states (count): 5
Argmin state(s) (ties): ['p5']

```

```

P(a*|p_i) on visited states:

```

```

phi_src
p1 0.831169
p2 0.752475
p3 0.708191
p4 0.673214
p5 0.625592

```

```

=====
Interpretation (assignment prompts)
=====

```

```

* Is the system strongly optimal?
  Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
  Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
  choices.
* Is behavior consistent across states?
  No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

Diagnostics: spread(max-min P)=0.205576, std(P)=0.078481

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.008812

a2 0.008528

a3 0.000284

L1 sum of abs diffs (actions): 0.017624

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.008812

p2 0.006538

p3 0.004832

p4 0.002274

p5 0.008243

L1 sum: 0.030699

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.009949

p2 0.006538

p3 0.005401

p4 0.002843

p5 0.008243

L1 sum: 0.032973

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.011696

a2 0.006800

a3 0.026564

L1 sum (per-action |D|): 0.045060

Global penalty rate |D|: data=0.260659 sim=0.253837 |D|=0.006822

=====
main: Engin-vsa_dataset_full.csv (trial 1, removed 704) - Tasks 1--4 finished
=====

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.198919)

epsilon (Task 3) : e = 0.374408 (min P(a*|p) = 0.625592)

simulation L1 : actions=0.0176 phi_src=0.0307 phi_dst=0.0330

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 2, removed 704))
=====

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====

alpha a1 a2 a3

phi_src

p1 467 63 31

p2 526 125 54

p3 403 133 47

p4 355 131 46

p5 271 109 53

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5

phi_src

668

718

```
p1 182 186 32 32 35
p2 153 152 154 39 28
p3 29 129 102 114 29
p4 16 33 103 100 103
p5 20 27 18 103 103
```

```
=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 25 25 5 6 2
p2 34 41 35 6 9
p3 6 45 39 37 6
p4 14 16 38 24 39
p5 12 11 9 35 42
```

```
=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 15 7 2 4 3
p2 20 14 16 2 2
p3 4 14 16 11 2
p4 4 2 16 10 14
p5 6 6 3 23 15
```

```
=====
N(alpha_k, beta) -- raw counts
=====
```

```
beta 0 1
alpha
a1 1612 410
a2 349 212
a3 115 116
```

```
=====
Policy P(alpha_k | phi_i)
=====
```

```
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.832442 0.112299 0.055258 1 PASS
p2 0.746099 0.177305 0.076596 1 PASS
p3 0.691252 0.228130 0.080617 1 PASS
p4 0.667293 0.246241 0.086466 1 PASS
p5 0.625866 0.251732 0.122402 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.389722 0.398287 0.068522 0.068522 0.074946 1 PASS
p2 0.290875 0.288973 0.292776 0.074144 0.053232 1 PASS
p3 0.071960 0.320099 0.253102 0.282878 0.071960 1 PASS
p4 0.045070 0.092958 0.290141 0.281690 0.290141 1 PASS
p5 0.073801 0.099631 0.066421 0.380074 0.380074 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.396825 0.396825 0.079365 0.095238 0.031746 1 PASS
p2 0.272000 0.328000 0.280000 0.048000 0.072000 1 PASS
p3 0.045113 0.338346 0.293233 0.278195 0.045113 1 PASS
p4 0.106870 0.122137 0.290076 0.183206 0.297710 1 PASS
p5 0.110092 0.100917 0.082569 0.321101 0.385321 1 PASS
```

768

```
=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====
```

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.483871 0.225806 0.064516 0.129032 0.096774 1 PASS
p2 0.370370 0.259259 0.296296 0.037037 0.037037 1 PASS
p3 0.085106 0.297872 0.340426 0.234043 0.042553 1 PASS
p4 0.086957 0.043478 0.347826 0.217391 0.304348 1 PASS
p5 0.113208 0.113208 0.056604 0.433962 0.283019 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.202770 0.202770 c_k matches empirical frequencies
a2 0.377897 0.377897 c_k matches empirical frequencies
a3 0.502165 0.502165 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.202770
a2 0.377897
a3 0.502165

alpha* = a1
c_* = 0.202770

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.202770 1 True
a2 0.377897 2 False
a3 0.502165 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.625866
e = 1 - min_i P(a*|p_i) = 0.374134
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.832442
p2 0.746099
p3 0.691252
p4 0.667293
p5 0.625866

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

818

Diagnostics: spread(max-min P)=0.206576, std(P)=0.079868

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.001421

a2 0.000569

a3 0.000853

L1 sum of abs diffs (actions): 0.002843

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.003127

p2 0.001990

p3 0.000284

p4 0.001990

p5 0.001137

L1 sum: 0.008528

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.004264

p2 0.002843

p3 0.000284

p4 0.002558

p5 0.001421

L1 sum: 0.011370

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.006681

a2 0.004718

a3 0.018980

L1 sum (per-action |D|): 0.030379

Global penalty rate |D|: data=0.260659 sim=0.258670 |D|=0.001990

=====
main: Engin-vsa_dataset_full.csv (trial 2, removed 704) - Tasks 1--4 finished
=====

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.202770)

epsilon (Task 3) : e = 0.374134 (min P(a*|p) = 0.625866)

simulation L1 : actions=0.0028 phi_src=0.0085 phi_dst=0.0114

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 3, removed 704))
=====

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====

alpha a1 a2 a3

phi_src

p1 461 68 34

p2 529 131 45

p3 411 125 37

p4 360 123 57

p5 281 103 49

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5

phi_src

918

```

p1 179 184 31 32 35
p2 154 153 156 36 30
p3 28 129 112 114 28
p4 20 28 102 107 103
p5 24 26 16 110 105

```

```

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====

```

```

phi_dst p1 p2 p3 p4 p5
phi_src
p1 24 29 5 8 2
p2 36 44 35 8 8
p3 9 44 36 32 4
p4 12 16 35 23 37
p5 12 9 8 39 35

```

```

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====

```

```

phi_dst p1 p2 p3 p4 p5
phi_src
p1 13 11 2 4 4
p2 16 13 12 1 3
p3 0 10 17 8 2
p4 5 3 19 13 17
p5 7 4 4 17 17

```

```

=====
N(alpha_k, beta) -- raw counts
=====

```

```

beta 0 1
alpha
a1 1625 417
a2 339 211
a3 110 112

```

```

=====
Policy P(alpha_k | phi_i)
=====

```

```

alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.818828 0.120782 0.060391 1 PASS
p2 0.750355 0.185816 0.063830 1 PASS
p3 0.717277 0.218150 0.064572 1 PASS
p4 0.666667 0.227778 0.105556 1 PASS
p5 0.648961 0.237875 0.113164 1 PASS

```

```

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====

```

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.388286 0.399132 0.067245 0.069414 0.075922 1 PASS
p2 0.291115 0.289225 0.294896 0.068053 0.056711 1 PASS
p3 0.068127 0.313869 0.272506 0.277372 0.068127 1 PASS
p4 0.055556 0.077778 0.283333 0.297222 0.286111 1 PASS
p5 0.085409 0.092527 0.056940 0.391459 0.373665 1 PASS

```

968

```

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====

```

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.352941 0.426471 0.073529 0.117647 0.029412 1 PASS
p2 0.274809 0.335878 0.267176 0.061069 0.061069 1 PASS
p3 0.072000 0.352000 0.288000 0.256000 0.032000 1 PASS
p4 0.097561 0.130081 0.284553 0.186992 0.300813 1 PASS
p5 0.116505 0.087379 0.077670 0.378641 0.339806 1 PASS

```

```

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

```

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.382353 0.323529 0.058824 0.117647 0.117647 1 PASS
p2 0.355556 0.288889 0.266667 0.022222 0.066667 1 PASS
p3 0.000000 0.270270 0.459459 0.216216 0.054054 1 PASS
p4 0.087719 0.052632 0.333333 0.228070 0.298246 1 PASS
p5 0.142857 0.081633 0.081633 0.346939 0.346939 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.204212 0.204212 c_k matches empirical frequencies
a2 0.383636 0.383636 c_k matches empirical frequencies
a3 0.504505 0.504505 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.204212
a2 0.383636
a3 0.504505

alpha* = a1
c_* = 0.204212

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.204212 1 True
a2 0.383636 2 False
a3 0.504505 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.648961
e = 1 - min_i P(a*|p_i) = 0.351039
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.818828
p2 0.750355
p3 0.717277
p4 0.666667
p5 0.648961

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
Partially - some variation across states but not extreme.

```

1018

```

Diagnostics: spread(max-min P)=0.169867, std(P)=0.068165

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====
Real rows: 3518 | Synthetic rows: 3518

1068 |P_data - P_sim| - action distribution
alpha
a1 0.006822
a2 0.007106
a3 0.000284
    L1 sum of abs diffs (actions): 0.014213

|P_data - P_sim| - phi_src visitation
phi_src
p1 0.000569
p2 0.002274
p3 0.003695
p4 0.000569
p5 0.002558
    L1 sum: 0.009665

|P_data - P_sim| - phi_dst visitation
phi_dst
p1 0.001706
p2 0.003411
p3 0.003695
p4 0.000000
p5 0.001990
    L1 sum: 0.010802

|mean_beta_data - mean_beta_sim| per action (penalty rate)
alpha
a1 0.006912
a2 0.006888
a3 0.005201
    L1 sum (per-action |D|): 0.019001

Global penalty rate |D|: data=0.260659 sim=0.256111 |D|=0.004548

=====
main: Engin-vsa_dataset_full.csv (trial 3, removed 704) - Tasks 1--4 finished
=====
rows processed : 2814
optimal action : alpha* = a1 (c* = 0.204212)
epsilon (Task 3) : e = 0.351039 (min P(a*|p) = 0.648961)
simulation L1 : actions=0.0142 phi_src=0.0097 phi_dst=0.0108

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 4, removed 704))
=====
Rows in dataset : 2814
States observed : ['p1', 'p2', 'p3', 'p4', 'p5']
Actions observed: ['a1', 'a2', 'a3']

=====
1118 N(phi_i, alpha_k) -- raw counts
=====
alpha a1 a2 a3
phi_src
p1 448 65 36
p2 530 128 46
p3 405 122 44
p4 378 130 48
p5 273 108 53

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src

```

```
p1 179 174 32 31 32
p2 164 160 146 34 26
p3 29 136 108 102 30
p4 21 35 108 106 108
p5 24 25 15 108 101
```

```
=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 25 25 6 7 2
p2 37 42 34 8 7
p3 9 37 34 35 7
p4 13 14 37 26 40
p5 13 10 8 40 37
```

```
=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
```

```
phi_dst p1 p2 p3 p4 p5
phi_src
p1 16 11 2 4 3
p2 17 12 13 1 3
p3 3 11 16 11 3
p4 3 3 15 14 13
p5 6 7 2 20 18
```

```
=====
N(alpha_k, beta) -- raw counts
=====
```

```
beta 0 1
alpha
a1 1632 402
a2 339 214
a3 105 122
```

```
=====
Policy P(alpha_k | phi_i)
=====
```

```
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.816029 0.118397 0.065574 1 PASS
p2 0.752841 0.181818 0.065341 1 PASS
p3 0.709282 0.213660 0.077058 1 PASS
p4 0.679856 0.233813 0.086331 1 PASS
p5 0.629032 0.248848 0.122120 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.399554 0.388393 0.071429 0.069196 0.071429 1 PASS
p2 0.309434 0.301887 0.275472 0.064151 0.049057 1 PASS
p3 0.071605 0.335802 0.266667 0.251852 0.074074 1 PASS
p4 0.055556 0.092593 0.285714 0.280423 0.285714 1 PASS
p5 0.087912 0.091575 0.054945 0.395604 0.369963 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
```

```
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.384615 0.384615 0.092308 0.107692 0.030769 1 PASS
p2 0.289062 0.328125 0.265625 0.062500 0.054688 1 PASS
p3 0.073770 0.303279 0.278689 0.286885 0.057377 1 PASS
p4 0.100000 0.107692 0.284615 0.200000 0.307692 1 PASS
p5 0.120370 0.092593 0.074074 0.370370 0.342593 1 PASS
```

```
=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====
```

1168

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.444444 0.305556 0.055556 0.111111 0.083333 1 PASS
p2 0.369565 0.260870 0.282609 0.021739 0.065217 1 PASS
p3 0.068182 0.250000 0.363636 0.250000 0.068182 1 PASS
p4 0.062500 0.062500 0.312500 0.291667 0.270833 1 PASS
p5 0.113208 0.132075 0.037736 0.377358 0.339623 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
1218 alpha penalty
a1 0.197640 0.197640 c_k matches empirical frequencies
a2 0.386980 0.386980 c_k matches empirical frequencies
a3 0.537445 0.537445 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.197640
a2 0.386980
a3 0.537445

alpha* = a1
c_* = 0.197640

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.197640 1 True
a2 0.386980 2 False
a3 0.537445 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.629032
e = 1 - min_i P(a*|p_i) = 0.370968
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.816029
p2 0.752841
p3 0.709282
p4 0.679856
p5 0.629032

1268
=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
Partially - some variation across states but not extreme.

```


Diagnostics: spread(max-min P)=0.186997, std(P)=0.071183

Task 4 - Simulation (source: estimation-result, seed=42)

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.006538

a2 0.008243

a3 0.001706

L1 sum of abs diffs (actions): 0.016487

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.011939

p2 0.001706

p3 0.005685

p4 0.004548

p5 0.003411

L1 sum: 0.027288

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.013360

p2 0.001137

p3 0.007106

p4 0.003980

p5 0.003411

L1 sum: 0.028994

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.014610

a2 0.012314

a3 0.047912

L1 sum (per-action |D|): 0.074836

1318 Global penalty rate |D|: data=0.260659 sim=0.255259 |D|=0.005401

main: Engin-vsa_dataset_full.csv (trial 4, removed 704) - Tasks 1--4 finished

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.197640)

epsilon (Task 3) : e = 0.370968 (min P(a*|p) = 0.629032)

simulation L1 : actions=0.0165 phi_src=0.0273 phi_dst=0.0290

Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 5, removed 704))

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

N(phi_i, alpha_k) -- raw counts

alpha a1 a2 a3

phi_src

p1 466 60 27

p2 548 127 49

p3 419 124 44

p4 347 122 55

p5 258 116 52

N(phi_i, alpha_k=a1, phi_j) -- raw counts

phi_dst p1 p2 p3 p4 p5

phi_src

1368

```

p1 179 191 30 31 35
p2 162 160 160 37 29
p3 30 136 110 112 31
p4 22 25 108 87 105
p5 19 25 16 104 94

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 22 25 7 5 1
p2 36 40 37 8 6
p3 8 40 33 37 6
p4 13 15 33 26 35
p5 13 10 10 45 38

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 9 9 3 3 3
p2 17 13 17 1 1
p3 2 12 14 13 3
p4 3 3 17 15 17
p5 7 6 4 19 16

=====
N(alpha_k, beta) -- raw counts
=====
beta 0 1
alpha
a1 1635 403
a2 346 203
a3 108 119

=====
Policy P(alpha_k | phi_i)
=====
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.842676 0.108499 0.048825 1 PASS
p2 0.756906 0.175414 0.067680 1 PASS
p3 0.713799 0.211244 0.074957 1 PASS
p4 0.662214 0.232824 0.104962 1 PASS
p5 0.605634 0.272300 0.122066 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.384120 0.409871 0.064378 0.066524 0.075107 1 PASS
p2 0.295620 0.291971 0.291971 0.067518 0.052920 1 PASS
p3 0.071599 0.324582 0.262530 0.267303 0.073986 1 PASS
p4 0.063401 0.072046 0.311239 0.250720 0.302594 1 PASS
p5 0.073643 0.096899 0.062016 0.403101 0.364341 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.366667 0.416667 0.116667 0.083333 0.016667 1 PASS
p2 0.283465 0.314961 0.291339 0.062992 0.047244 1 PASS
p3 0.064516 0.322581 0.266129 0.298387 0.048387 1 PASS
p4 0.106557 0.122951 0.270492 0.213115 0.286885 1 PASS
p5 0.112069 0.086207 0.086207 0.387931 0.327586 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

```

1418

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.333333 0.333333 0.111111 0.111111 0.111111 1 PASS
p2 0.346939 0.265306 0.346939 0.020408 0.020408 1 PASS
p3 0.045455 0.272727 0.318182 0.295455 0.068182 1 PASS
p4 0.054545 0.054545 0.309091 0.272727 0.309091 1 PASS
p5 0.134615 0.115385 0.076923 0.365385 0.307692 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.197743 0.197743 c_k matches empirical frequencies
a2 0.369763 0.369763 c_k matches empirical frequencies
a3 0.524229 0.524229 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.197743
a2 0.369763
a3 0.524229

alpha* = a1
c_* = 0.197743

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.197743 1 True
a2 0.369763 2 False
a3 0.524229 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.605634
e = 1 - min_i P(a*|p_i) = 0.394366
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.842676
p2 0.756906
p3 0.713799
p4 0.662214
p5 0.605634

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

1468

```

Diagnostics: spread(max-min P)=0.237043, std(P)=0.090552

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====
Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution
alpha
a1 0.004264
a2 0.003980
a3 0.000284
  L1 sum of abs diffs (actions): 0.008528

|P_data - P_sim| - phi_src visitation
phi_src
p1 0.000569
p2 0.002558
p3 0.007391
p4 0.003411
p5 0.001990
  L1 sum: 0.015918

|P_data - P_sim| - phi_dst visitation
phi_dst
p1 0.001990
p2 0.003411
p3 0.007106
p4 0.003980
p5 0.001706
  L1 sum: 0.018192

|mean_beta_data - mean_beta_sim| per action (penalty rate)
alpha
a1 0.015202
a2 0.011430
a3 0.026564
  L1 sum (per-action |D|): 0.053196

Global penalty rate |D|: data=0.260659 sim=0.248721 |D|=0.011939

=====
main: Engin-vsa_dataset_full.csv (trial 5, removed 704) - Tasks 1--4 finished
=====
rows processed : 2814
optimal action : alpha* = a1 (c* = 0.197743)
epsilon (Task 3) : e = 0.394366 (min P(a*|p) = 0.605634)
simulation L1 : actions=0.0085 phi_src=0.0159 phi_dst=0.0182

=====
Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 6, removed 704))
=====
Rows in dataset : 2814
States observed : ['p1', 'p2', 'p3', 'p4', 'p5']
Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====
alpha a1 a2 a3
phi_src
p1 452 64 32
p2 539 123 46
p3 395 123 46
p4 371 124 52
p5 288 112 47

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src

```

p1 181 174 32 31 34
p2 160 160 156 37 26
p3 26 131 102 111 25
p4 23 33 103 105 107
p5 26 24 20 106 112

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 24 25 8 6 1
p2 35 43 31 7 7
p3 8 42 34 32 7
p4 11 17 34 24 38
p5 13 12 8 39 40

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 11 12 3 2 4
p2 17 10 15 1 3
p3 3 11 19 10 3
p4 4 3 13 15 17
p5 7 5 3 16 16

=====
N(alpha_k, beta) -- raw counts
=====

beta 0 1
alpha
a1 1627 418
a2 339 207
a3 108 115

=====
Policy P(alpha_k | phi_i)
=====

alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.824818 0.116788 0.058394 1 PASS
p2 0.761299 0.173729 0.064972 1 PASS
p3 0.700355 0.218085 0.081560 1 PASS
p4 0.678245 0.226691 0.095064 1 PASS
p5 0.644295 0.250559 0.105145 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.400442 0.384956 0.070796 0.068584 0.075221 1 PASS
p2 0.296846 0.296846 0.289425 0.068646 0.048237 1 PASS
p3 0.065823 0.331646 0.258228 0.281013 0.063291 1 PASS
p4 0.061995 0.088949 0.277628 0.283019 0.288410 1 PASS
p5 0.090278 0.083333 0.069444 0.368056 0.388889 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.375000 0.390625 0.125000 0.093750 0.015625 1 PASS
p2 0.284553 0.349593 0.252033 0.056911 0.056911 1 PASS
p3 0.065041 0.341463 0.276423 0.260163 0.056911 1 PASS
p4 0.088710 0.137097 0.274194 0.193548 0.306452 1 PASS
p5 0.116071 0.107143 0.071429 0.348214 0.357143 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

1618

1668

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.343750 0.375000 0.093750 0.062500 0.125000 1 PASS
p2 0.369565 0.217391 0.326087 0.021739 0.065217 1 PASS
p3 0.065217 0.239130 0.413043 0.217391 0.065217 1 PASS
p4 0.076923 0.057692 0.250000 0.288462 0.326923 1 PASS
p5 0.148936 0.106383 0.063830 0.340426 0.340426 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.204401 0.204401 c_k matches empirical frequencies
a2 0.379121 0.379121 c_k matches empirical frequencies
a3 0.515695 0.515695 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.204401
a2 0.379121
a3 0.515695

alpha* = a1
c_* = 0.204401

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.204401 1 True
a2 0.379121 2 False
a3 0.515695 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.644295
e = 1 - min_i P(a*|p_i) = 0.355705
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.824818
p2 0.761299
p3 0.700355
p4 0.678245
p5 0.644295

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
Partially - some variation across states but not extreme.

```

```

Diagnostics: spread(max-min P)=0.180522, std(P)=0.071653

=====
1718 Task 4 - Simulation (source: estimation-result, seed=42)
=====
Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution
alpha
a1 0.009949
a2 0.009949
a3 0.000000
  L1 sum of abs diffs (actions): 0.019898

|P_data - P_sim| - phi_src visitation
phi_src
p1 0.009096
p2 0.003695
p3 0.005401
p4 0.007675
p5 0.000284
  L1 sum: 0.026151

|P_data - P_sim| - phi_dst visitation
phi_dst
p1 0.010517
p2 0.002843
p3 0.005969
p4 0.007959
p5 0.000569
  L1 sum: 0.027857

|mean_beta_data - mean_beta_sim| per action (penalty rate)
alpha
a1 0.005797
a2 0.003629
a3 0.024648
  L1 sum (per-action |D|): 0.034074

Global penalty rate |D|: data=0.260659 sim=0.257248 |D|=0.003411

=====
main: Engin-vsa_dataset_full.csv (trial 6, removed 704) - Tasks 1--4 finished
=====
rows processed : 2814
optimal action : alpha* = a1 (c* = 0.204401)
epsilon (Task 3) : e = 0.355705 (min P(a*|p) = 0.644295)
simulation L1 : actions=0.0199 phi_src=0.0262 phi_dst=0.0279

=====
1768 Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 7, removed 704))
=====
Rows in dataset : 2814
States observed : ['p1', 'p2', 'p3', 'p4', 'p5']
Actions observed: ['a1', 'a2', 'a3']

=====
N(phi_i, alpha_k) -- raw counts
=====
alpha a1 a2 a3
phi_src
p1 463 63 30
p2 546 124 44
p3 404 127 42
p4 361 125 57
p5 274 104 50

=====
N(phi_i, alpha_k=a1, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src

```

1818

```

p1 180 183 32 31 37
p2 165 153 162 37 29
p3 27 143 101 109 24
p4 25 35 103 101 97
p5 20 32 18 108 96

=====
N(phi_i, alpha_k=a2, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 20 29 3 9 2
p2 34 42 32 8 8
p3 7 41 36 36 7
p4 12 16 32 29 36
p5 11 10 8 38 37

=====
N(phi_i, alpha_k=a3, phi_j) -- raw counts
=====
phi_dst p1 p2 p3 p4 p5
phi_src
p1 11 9 3 3 4
p2 14 11 15 1 3
p3 2 13 16 10 1
p4 4 3 16 17 17
p5 7 7 3 19 14

=====
N(alpha_k, beta) -- raw counts
=====
beta 0 1
alpha
a1 1633 415
a2 338 205
a3 105 118

=====
Policy P(alpha_k | phi_i)
=====
alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.832734 0.113309 0.053957 1 PASS
p2 0.764706 0.173669 0.061625 1 PASS
p3 0.705061 0.221640 0.073298 1 PASS
p4 0.664825 0.230203 0.104972 1 PASS
p5 0.640187 0.242991 0.116822 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a1)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.388769 0.395248 0.069114 0.066955 0.079914 1 PASS
p2 0.302198 0.280220 0.296703 0.067766 0.053114 1 PASS
p3 0.066832 0.353960 0.250000 0.269802 0.059406 1 PASS
p4 0.069252 0.096953 0.285319 0.279778 0.268698 1 PASS
p5 0.072993 0.116788 0.065693 0.394161 0.350365 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a2)
=====
phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.317460 0.460317 0.047619 0.142857 0.031746 1 PASS
p2 0.274194 0.338710 0.258065 0.064516 0.064516 1 PASS
p3 0.055118 0.322835 0.283465 0.283465 0.055118 1 PASS
p4 0.096000 0.128000 0.256000 0.232000 0.288000 1 PASS
p5 0.105769 0.096154 0.076923 0.365385 0.355769 1 PASS

=====
Transition P(phi_j | phi_i, alpha_k=a3)
=====

```


1868

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.366667 0.300000 0.100000 0.100000 0.133333 1 PASS
p2 0.318182 0.250000 0.340909 0.022727 0.068182 1 PASS
p3 0.047619 0.309524 0.380952 0.238095 0.023810 1 PASS
p4 0.070175 0.052632 0.280702 0.298246 0.298246 1 PASS
p5 0.140000 0.140000 0.060000 0.380000 0.280000 1 PASS

```

```

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====

```

```

c_k empirical Cons.
alpha penalty
a1 0.202637 0.202637 c_k matches empirical frequencies
a2 0.377532 0.377532 c_k matches empirical frequencies
a3 0.529148 0.529148 c_k matches empirical frequencies

```

```

=====
Consistency checks
=====

```

```

[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

```

```

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====

```

```

Penalty probabilities c_k = P(beta = 1 | alpha_k):

```

```

alpha
a1 0.202637
a2 0.377532
a3 0.529148

```

```

alpha* = a1
c_* = 0.202637

```

```

=====
Ranking by c_k (ascending)
=====

```

```

c_k rank is_optimal
alpha
a1 0.202637 1 True
a2 0.377532 2 False
a3 0.529148 3 False

```

```

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====

```

```

a* = a1
min_i P(a*|p_i) over visited states = 0.640187
e = 1 - min_i P(a*|p_i) = 0.359813
Visited states (count): 5
Argmin state(s) (ties): ['p5']

```

```

P(a*|p_i) on visited states:

```

```

phi_src
p1 0.832734
p2 0.764706
p3 0.705061
p4 0.664825
p5 0.640187

```

1918

```

=====
Interpretation (assignment prompts)
=====

```

```

* Is the system strongly optimal?
  Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
  Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
  choices.
* Is behavior consistent across states?
  Partially - some variation across states but not extreme.

```

Diagnostics: spread(max-min P)=0.192547, std(P)=0.077996

Task 4 - Simulation (source: estimation-result, seed=42)

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.008528

a2 0.008243

a3 0.000284

L1 sum of abs diffs (actions): 0.017055

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.005117

p2 0.010802

p3 0.000853

p4 0.007391

p5 0.007675

L1 sum: 0.031836

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.006822

p2 0.009096

p3 0.000569

p4 0.007391

p5 0.007959

L1 sum: 0.031836

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.006585

a2 0.006006

a3 0.033631

L1 sum (per-action |D|): 0.046222

Global penalty rate |D|: data=0.260659 sim=0.255827 |D|=0.004832

main: Engin-vsa_dataset_full.csv (trial 7, removed 704) - Tasks 1--4 finished

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.202637)

epsilon (Task 3) : e = 0.359813 (min P(a*|p) = 0.640187)

simulation L1 : actions=0.0171 phi_src=0.0318 phi_dst=0.0318

Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 8, removed 704))

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

N(phi_i, alpha_k) -- raw counts

alpha a1 a2 a3

phi_src

p1 445 58 31

p2 545 130 47

p3 402 130 49

p4 375 126 53

p5 265 111 47

N(phi_i, alpha_k=a1, phi_j) -- raw counts

phi_dst p1 p2 p3 p4 p5

phi_src

1968

2018

p1 178 174 31 29 33
 p2 160 159 160 39 27
 p3 23 133 112 106 28
 p4 24 31 106 102 112
 p5 22 26 17 107 93

=====

N(phi_i, alpha_k=a2, phi_j) -- raw counts

=====

phi_dst p1 p2 p3 p4 p5
 phi_src
 p1 20 22 7 7 2
 p2 38 43 35 7 7
 p3 8 41 36 38 7
 p4 10 16 38 28 34
 p5 15 12 8 38 38

=====

N(phi_i, alpha_k=a3, phi_j) -- raw counts

=====

phi_dst p1 p2 p3 p4 p5
 phi_src
 p1 13 9 3 4 2
 p2 17 14 13 0 3
 p3 4 13 16 13 3
 p4 4 3 17 14 15
 p5 5 5 3 19 15

=====

N(alpha_k, beta) -- raw counts

=====

beta 0 1
 alpha
 a1 1632 400
 a2 343 212
 a3 109 118

=====

Policy P(alpha_k | phi_i)

=====

alpha a1 a2 a3 SUM Cons.
 phi_src
 p1 0.833333 0.108614 0.058052 1 PASS
 p2 0.754848 0.180055 0.065097 1 PASS
 p3 0.691910 0.223752 0.084337 1 PASS
 p4 0.676895 0.227437 0.095668 1 PASS
 p5 0.626478 0.262411 0.111111 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a1)

=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
 phi_src
 p1 0.400000 0.391011 0.069663 0.065169 0.074157 1 PASS
 p2 0.293578 0.291743 0.293578 0.071560 0.049541 1 PASS
 p3 0.057214 0.330846 0.278607 0.263682 0.069652 1 PASS
 p4 0.064000 0.082667 0.282667 0.272000 0.298667 1 PASS
 p5 0.083019 0.098113 0.064151 0.403774 0.350943 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a2)

=====

2068

phi_dst p1 p2 p3 p4 p5 SUM Cons.
 phi_src
 p1 0.344828 0.379310 0.120690 0.120690 0.034483 1 PASS
 p2 0.292308 0.330769 0.269231 0.053846 0.053846 1 PASS
 p3 0.061538 0.315385 0.276923 0.292308 0.053846 1 PASS
 p4 0.079365 0.126984 0.301587 0.222222 0.269841 1 PASS
 p5 0.135135 0.108108 0.072072 0.342342 0.342342 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a3)

=====

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.419355 0.290323 0.096774 0.129032 0.064516 1 PASS
p2 0.361702 0.297872 0.276596 0.000000 0.063830 1 PASS
p3 0.081633 0.265306 0.326531 0.265306 0.061224 1 PASS
p4 0.075472 0.056604 0.320755 0.264151 0.283019 1 PASS
p5 0.106383 0.106383 0.063830 0.404255 0.319149 1 PASS

```

```

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====

```

```

c_k empirical Cons.
alpha penalty
a1 0.196850 0.196850 c_k matches empirical frequencies
a2 0.381982 0.381982 c_k matches empirical frequencies
a3 0.519824 0.519824 c_k matches empirical frequencies

```

```

=====
Consistency checks
=====

```

```

[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

```

```

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====

```

```

Penalty probabilities c_k = P(beta = 1 | alpha_k):

```

```

alpha
a1 0.196850
a2 0.381982
a3 0.519824

```

```

alpha* = a1
c_* = 0.196850

```

```

=====
Ranking by c_k (ascending)
=====

```

```

c_k rank is_optimal
alpha
a1 0.196850 1 True
a2 0.381982 2 False
a3 0.519824 3 False

```

```

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====

```

```

a* = a1
min_i P(a*|p_i) over visited states = 0.626478
e = 1 - min_i P(a*|p_i) = 0.373522
Visited states (count): 5
Argmin state(s) (ties): ['p5']

```

```

P(a*|p_i) on visited states:

```

```

phi_src
p1 0.833333
p2 0.754848
p3 0.691910
p4 0.676895
p5 0.626478

```

```

=====
Interpretation (assignment prompts)
=====

```

```

* Is the system strongly optimal?
  Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
  Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
  choices.
* Is behavior consistent across states?
  No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

2118

Diagnostics: spread(max-min P)=0.206856, std(P)=0.079683

Task 4 - Simulation (source: estimation-result, seed=42)

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.003411

a2 0.004832

a3 0.001421

L1 sum of abs diffs (actions): 0.009665

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.004548

p2 0.002843

p3 0.003695

p4 0.001137

p5 0.004264

L1 sum: 0.016487

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.005969

p2 0.004264

p3 0.003411

p4 0.000853

p5 0.004264

L1 sum: 0.018761

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.014596

a2 0.005228

a3 0.018727

L1 sum (per-action |D|): 0.038550

Global penalty rate |D|: data=0.260659 sim=0.252132 |D|=0.008528

main: Engin-vsa_dataset_full.csv (trial 8, removed 704) - Tasks 1--4 finished

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.196850)

epsilon (Task 3) : e = 0.373522 (min P(a*|p) = 0.626478)

simulation L1 : actions=0.0097 phi_src=0.0165 phi_dst=0.0188

Task 1 - Estimation (dataset: Engin-vsa_dataset_full.csv (trial 9, removed 704))

Rows in dataset : 2814

States observed : ['p1', 'p2', 'p3', 'p4', 'p5']

Actions observed: ['a1', 'a2', 'a3']

N(phi_i, alpha_k) -- raw counts

alpha a1 a2 a3

phi_src

p1 456 63 28

p2 537 122 47

p3 419 125 47

p4 358 121 49

p5 279 112 51

N(phi_i, alpha_k=a1, phi_j) -- raw counts

phi_dst p1 p2 p3 p4 p5

phi_src

p1 184 177 33 29 33
p2 157 167 149 36 28
p3 25 135 114 113 32
p4 22 31 101 105 99
p5 26 27 16 106 104

=====

N(phi_i, alpha_k=a2, phi_j) -- raw counts

=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 19 27 8 7 2
p2 33 46 32 5 6
p3 7 43 30 40 5
p4 11 11 36 25 38
p5 14 11 10 39 38

=====

N(phi_i, alpha_k=a3, phi_j) -- raw counts

=====

phi_dst p1 p2 p3 p4 p5
phi_src
p1 13 6 1 4 4
p2 15 13 15 1 3
p3 3 14 15 13 2
p4 5 2 15 14 13
p5 5 4 2 21 19

=====

N(alpha_k, beta) -- raw counts

=====

beta 0 1
alpha
a1 1642 407
a2 341 202
a3 108 114

=====

Policy P(alpha_k | phi_i)

=====

alpha a1 a2 a3 SUM Cons.
phi_src
p1 0.833638 0.115174 0.051188 1 PASS
p2 0.760623 0.172805 0.066572 1 PASS
p3 0.708968 0.211506 0.079526 1 PASS
p4 0.678030 0.229167 0.092803 1 PASS
p5 0.631222 0.253394 0.115385 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a1)

=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.403509 0.388158 0.072368 0.063596 0.072368 1 PASS
p2 0.292365 0.310987 0.277467 0.067039 0.052142 1 PASS
p3 0.059666 0.322196 0.272076 0.269690 0.076372 1 PASS
p4 0.061453 0.086592 0.282123 0.293296 0.276536 1 PASS
p5 0.093190 0.096774 0.057348 0.379928 0.372760 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a2)

=====

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.301587 0.428571 0.126984 0.111111 0.031746 1 PASS
p2 0.270492 0.377049 0.262295 0.040984 0.049180 1 PASS
p3 0.056000 0.344000 0.240000 0.320000 0.040000 1 PASS
p4 0.090909 0.090909 0.297521 0.206612 0.314050 1 PASS
p5 0.125000 0.098214 0.089286 0.348214 0.339286 1 PASS

=====

Transition P(phi_j | phi_i, alpha_k=a3)

=====

2268

2318

```

phi_dst p1 p2 p3 p4 p5 SUM Cons.
phi_src
p1 0.464286 0.214286 0.035714 0.142857 0.142857 1 PASS
p2 0.319149 0.276596 0.319149 0.021277 0.063830 1 PASS
p3 0.063830 0.297872 0.319149 0.276596 0.042553 1 PASS
p4 0.102041 0.040816 0.306122 0.285714 0.265306 1 PASS
p5 0.098039 0.078431 0.039216 0.411765 0.372549 1 PASS

=====
Penalty probabilities c_k = P(beta=1 | alpha_k)
=====
c_k empirical Cons.
alpha penalty
a1 0.198633 0.198633 c_k matches empirical frequencies
a2 0.372007 0.372007 c_k matches empirical frequencies
a3 0.513514 0.513514 c_k matches empirical frequencies

=====
Consistency checks
=====
[PASS] sum_k P(alpha_k | phi_i) == 1
[PASS] sum_j P(phi_j | phi_i, alpha_k) == 1
[PASS] c_k matches empirical frequencies

=====
Task 2 - Optimal Action Identification (source: estimation-result)
=====
Penalty probabilities c_k = P(beta = 1 | alpha_k):
alpha
a1 0.198633
a2 0.372007
a3 0.513514

alpha* = a1
c_* = 0.198633

=====
Ranking by c_k (ascending)
=====
c_k rank is_optimal
alpha
a1 0.198633 1 True
a2 0.372007 2 False
a3 0.513514 3 False

=====
Task 3 - e-optimality (source: estimation-result + optimal-result)
=====
a* = a1
min_i P(a*|p_i) over visited states = 0.631222
e = 1 - min_i P(a*|p_i) = 0.368778
Visited states (count): 5
Argmin state(s) (ties): ['p5']

P(a*|p_i) on visited states:
phi_src
p1 0.833638
p2 0.760623
p3 0.708968
p4 0.678030
p5 0.631222

=====
Interpretation (assignment prompts)
=====
* Is the system strongly optimal?
Weakly - e is large: at least one visited state rarely chooses a* under the empirical policy.
* Is it noisy?
Yes - high e and/or high spread across states suggests heterogeneous or stochastic-looking action
choices.
* Is behavior consistent across states?
No - behavior differs markedly across visited states (large spread in P(a*|p)).

```

Diagnostics: spread(max-min P)=0.202416, std(P)=0.077938

=====
Task 4 - Simulation (source: estimation-result, seed=42)
=====

Real rows: 3518 | Synthetic rows: 3518

|P_data - P_sim| - action distribution

alpha

a1 0.010517

a2 0.009096

a3 0.001421

L1 sum of abs diffs (actions): 0.021035

|P_data - P_sim| - phi_src visitation

phi_src

p1 0.003411

p2 0.006254

p3 0.009096

p4 0.001421

p5 0.001990

L1 sum: 0.022172

|P_data - P_sim| - phi_dst visitation

phi_dst

p1 0.005117

p2 0.004832

p3 0.009380

p4 0.001137

p5 0.001706

L1 sum: 0.022172

|mean_beta_data - mean_beta_sim| per action (penalty rate)

alpha

a1 0.013678

a2 0.001211

a3 0.016445

L1 sum (per-action |D|): 0.031334

Global penalty rate |D|: data=0.260659 sim=0.249574 |D|=0.011086

=====
main: Engin-vsa_dataset_full.csv (trial 9, removed 704) - Tasks 1--4 finished
=====

rows processed : 2814

optimal action : alpha* = a1 (c* = 0.198633)

epsilon (Task 3) : e = 0.368778 (min P(a*|p) = 0.631222)

simulation L1 : actions=0.0210 phi_src=0.0222 phi_dst=0.0222

=====
Task 5 - Perturbation (20% rows removed, 10 trial(s))
=====

Dataset : D:\Documents\00-PhD\04-CSE8003\assignments\term_project\Engin-vsa_dataset_full.csv

Rows : 3518 full

Baseline (full data, Tasks 1--4)

alpha* = a1 c* = 0.199608

epsilon = 0.370166

Perturbed runs

Engin-vsa_dataset_full.csv (trial 0, removed 704): alpha*=a1 epsilon=0.391705 L1_act=0.0057

Engin-vsa_dataset_full.csv (trial 1, removed 704): alpha*=a1 epsilon=0.374408 L1_act=0.0176

Engin-vsa_dataset_full.csv (trial 2, removed 704): alpha*=a1 epsilon=0.374134 L1_act=0.0028

Engin-vsa_dataset_full.csv (trial 3, removed 704): alpha*=a1 epsilon=0.351039 L1_act=0.0142

Engin-vsa_dataset_full.csv (trial 4, removed 704): alpha*=a1 epsilon=0.370968 L1_act=0.0165

Engin-vsa_dataset_full.csv (trial 5, removed 704): alpha*=a1 epsilon=0.394366 L1_act=0.0085

Engin-vsa_dataset_full.csv (trial 6, removed 704): alpha*=a1 epsilon=0.355705 L1_act=0.0199

Engin-vsa_dataset_full.csv (trial 7, removed 704): alpha*=a1 epsilon=0.359813 L1_act=0.0171

Engin-vsa_dataset_full.csv (trial 8, removed 704): alpha*=a1 epsilon=0.373522 L1_act=0.0097

Engin-vsa_dataset_full.csv (trial 9, removed 704): alpha*=a1 epsilon=0.368778 L1_act=0.0210

epsilon mean=0.371444 std=0.013996 delta_mean_vs_baseline=+0.001278


```

Spread across states per action (max_phi c_k - min_phi c_k):
  Baseline: a1=0.051272 a2=0.099706 a3=0.243992
  Perturbed: a1=mean 0.050590 std 0.010036 a2=mean 0.120915 std 0.018252 a3=mean 0.249805 std 0.040117

|c_k_per_state - baseline| max cell delta:
  mean=0.051845 std=0.021050 max=0.094017

=====
Task 6 - Model assumption check
=====
c_k(phi_i) = P(beta=1 | alpha_k, phi_i)

Engin-vsa_dataset_full.csv (baseline) (n=3518 rows):
alpha a1 a2 a3
phi_src
p1 0.205934 0.346154 0.461538
p2 0.225564 0.350318 0.362069
p3 0.195695 0.445860 0.517241
p4 0.174292 0.363636 0.606061
p5 0.178363 0.384058 0.603175

Engin-vsa_dataset_full.csv (trial 0, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.207506 0.358209 0.451613
p2 0.224953 0.351562 0.360000
p3 0.189526 0.459854 0.555556
p4 0.183333 0.359375 0.568627
p5 0.193182 0.398305 0.634615

Engin-vsa_dataset_full.csv (trial 1, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.191964 0.322581 0.448276
p2 0.225564 0.373134 0.390244
p3 0.195181 0.436508 0.488889
p4 0.183024 0.385246 0.606557
p5 0.185606 0.379630 0.620000

Engin-vsa_dataset_full.csv (trial 2, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.201285 0.380952 0.483871
p2 0.226236 0.344000 0.314815
p3 0.203474 0.451128 0.489362
p4 0.188732 0.351145 0.608696
p5 0.177122 0.357798 0.622642

Engin-vsa_dataset_full.csv (trial 3, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.219089 0.367647 0.382353
p2 0.219282 0.358779 0.377778
p3 0.214112 0.440000 0.567568
p4 0.175000 0.333333 0.596491
p5 0.174377 0.417476 0.551020

Engin-vsa_dataset_full.csv (trial 4, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.207589 0.323077 0.472222
p2 0.226415 0.312500 0.347826
p3 0.177778 0.467213 0.500000
p4 0.177249 0.400000 0.666667
p5 0.183150 0.407407 0.660377

Engin-vsa_dataset_full.csv (trial 5, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.208155 0.333333 0.555556
p2 0.222628 0.338583 0.408163
p3 0.195704 0.443548 0.500000
p4 0.161383 0.336066 0.618182

```

2468

2518

```
p5 0.178295 0.379310 0.538462

Engin-vsa_dataset_full.csv (trial 6, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.221239 0.375000 0.437500
p2 0.233766 0.341463 0.391304
p3 0.197468 0.447154 0.521739
p4 0.164420 0.354839 0.596154
p5 0.184028 0.375000 0.595745

Engin-vsa_dataset_full.csv (trial 7, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.205184 0.333333 0.466667
p2 0.230769 0.306452 0.386364
p3 0.193069 0.425197 0.500000
p4 0.171745 0.384000 0.614035
p5 0.197080 0.423077 0.620000

Engin-vsa_dataset_full.csv (trial 8, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.200000 0.293103 0.419355
p2 0.229358 0.353846 0.382979
p3 0.179104 0.438462 0.530612
p4 0.178667 0.357143 0.603774
p5 0.177358 0.423423 0.617021

Engin-vsa_dataset_full.csv (trial 9, removed 704) (n=2814 rows):
alpha a1 a2 a3
phi_src
p1 0.201754 0.301587 0.428571
p2 0.219739 0.344262 0.361702
p3 0.190931 0.440000 0.510638
p4 0.184358 0.371901 0.591837
p5 0.182796 0.366071 0.627451

Process finished with exit code 0
```